

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 23 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM
49 //
50 // ---- 常见优化手段速查清单 ----
51 //
52 // 1. Coalesced Memory Access (合并访问)
53 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
54 //   - 否则产生多次事务 (最坏 32 次)
55 //
56 // 2. Tiling (分块)
57 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
58 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
59 //
60 // 3. Vectorized Memory Access (向量化)
61 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数
62 //   - float4 = 128-bit, 单条指令加载 16 bytes

```

```

63 //
64 // 4. Thread Tile (寄存器分块)
65 //   - 每个线程计算多个输出元素 (TM×TN)，提高计算密度
66 //   - 减少线程总数，降低同步开销
67 //
68 // 5. Bank Conflict Avoidance
69 //   - Shared memory 有 32 banks × 4 bytes
70 //   - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
71 //   - 解决方案：PAD (在每行末尾加 1 个元素打破对齐)
72 //
73 // 6. Pipeline / Double Buffering (流水线)
74 //   - cp.async 异步拷贝下一批数据时同时做当前批的计算
75 //   - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
76 //
77 // 7. Tensor Core (MMA / WGMMMA)
78 //   - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
79 //   - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
80 //
81 // 8. Warp Specialization (Hopper+)
82 //   - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
83 //   - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
84 //
85 // 9. TMA (Tensor Memory Accelerator, Hopper+)
86 //   - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
87 //   - 配合 cp.async.bulk 实现异步数据搬运
88 //
89 // ---- Roofline 分析公式 ----
90 //
91 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
92 //
93 // GEMM (M=N=K=4096):
94 //   FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
95 //   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
96 //   AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
97 //   FP16 TC ≈ 295:1, FP32 ≈ 20:1)
98 //
99 // GEMV (M=4096, K=4096):
100 //   FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
101 //   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 //   AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105 //
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 //   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 //   - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //     注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 //   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115 //
116 #include <algorithm>
117 #include <cuda_fp16.h>
118 #include <cuda_runtime.h>
119 #include <float.h>
120 #include <stdio.h>
121 #include <stdlib.h>
122 #include <math.h>
123 #include <cublas_v2.h>
124 //
125 // WGMMMA/TMA headers (sm90a only)

```

```

126 #if defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900
127 #include <cuda/barrier>
128 #endif
129
130 #define WARP_SIZE 32
131 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
132 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
133 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
134
135 // =====
136 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
137 // =====
138
139 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
140 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
141 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
142 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
143 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
144 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
145 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
146 //
147 // 初始: 每个 lane 持有自己的值 v0..v7
148 // lane:  0    1    2    3    4    5    6    7
149 // val:  v0   v1   v2   v3   v4   v5   v6   v7
150 //
151 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
152 //
153 //      ┌───────────┐
154 // 对:   (0,4) (1,5) (2,6) (3,7)
155 //
156 // lane:  0    1    2    3    4    5    6    7
157 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
158 //
159 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
160 //
161 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
162 // 对:   (0,2) (1,3) (4,6) (5,7)
163 //      ─── 前4个一组 ───      ─── 后4个一组 ───
164 //
165 // lane:  0    1    2    3    4    5    6    7
166 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$  (每lane持有前一轮2个值的和再加本轮配对)
167 //      = v0+v2+v4+v6 ... (逐步归约)
168 //
169 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
170 //
171 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
172 // 对:   (0,1) (2,3) (4,5) (6,7)
173 //
174 // lane:  0    1    2    3    4    5    6    7
175 // val:  $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
176 //
177 // mask=0: 循环终止, 归约完成。
178 //
179 // 关键性质:
180 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
181 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
182 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
183 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
184 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
185 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
186
187 template <const int kWarpSize = WARP_SIZE, typename T = float>
188 __device__ __forceinline__ T warp_reduce_sum(T val) {
189 #pragma unroll
190     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
191         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
192     }
193 }

```

```

188     }
189     return val;
190 }
191
192 // Warp Reduce Max — generic
193 template <const int kWarpSize = WARP_SIZE, typename T = float>
194 __device__ __forceinline__ T warp_reduce_max(T val) {
195     #pragma unroll
196     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
197         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
198     }
199     return val;
200 }
201
202 // =====
203 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
204 // =====
205
206 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
207 // 两级归约流程:
208 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
209 // 2. warp leader (lane=0) 写入 shared memory
210 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
211 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
212 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
213 template <const int NUM_THREADS = 256>
214 __device__ float block_reduce_sum(float val) {
215     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
216     int warp = threadIdx.x / WARP_SIZE;
217     int lane = threadIdx.x % WARP_SIZE;
218     static __shared__ float shared[NUM_WARPS];
219
220     float value = warp_reduce_sum<WARP_SIZE>(val);
221     if (lane == 0)
222         shared[warp] = value;
223     __syncthreads();
224     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
225     value = warp_reduce_sum<NUM_WARPS>(value);
226     // 关键: broadcast 结果到所有线程, 后续用 result
227     // 做除法等操作时所有线程都能拿到
228     value = __shfl_sync(0xffffffff, value, 0, 32);
229     return value;
230 }
231
232 // Block Reduce Max — FP32 (增强版, 带 broadcast)
233 template <const int NUM_THREADS = 256>
234 __device__ float block_reduce_max(float val) {
235     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
236     int warp = threadIdx.x / WARP_SIZE;
237     int lane = threadIdx.x % WARP_SIZE;
238     static __shared__ float shared[NUM_WARPS];
239
240     float value = warp_reduce_max<WARP_SIZE>(val);
241     if (lane == 0)
242         shared[warp] = value;
243     __syncthreads();
244     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
245     value = warp_reduce_max<NUM_WARPS>(value);
246     value = __shfl_sync(0xffffffff, value, 0, 32);
247     return value;
248 }
249
250 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----

```

```

251 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
252 // 跨 block 求和的常见模式, 适合 N 较大时使用;
253 // Grid: ((N + 255) / 256, 1, 1)
254 // Block: (256, 1, 1)
255 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
256 template <const int NUM_THREADS = 256>
257 __global__ void block_reduce_all(float *a, float *y, int N) {
258     int tid = threadIdx.x;
259     int idx = blockIdx.x * NUM_THREADS + tid;
260     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
261     __shared__ float reduce_smem[NUM_WARPS];
262
263     float sum = (idx < N) ? a[idx] : 0.0f;
264     int warp = tid / WARP_SIZE;
265     int lane = tid % WARP_SIZE;
266
267     sum = warp_reduce_sum<WARP_SIZE>(sum);
268     if (lane == 0)
269         reduce_smem[warp] = sum;
270     __syncthreads();
271
272     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
273     if (warp == 0)
274         sum = warp_reduce_sum<NUM_WARPS>(sum);
275     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
276         atomicAdd(y, sum);
277 }
278
279 // ---- Dot Product: y = sum(a[i] * b[i]) ----
280 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
281 // Grid: ((N + 255) / 256, 1, 1)
282 // Block: (256, 1, 1)
283 // source: LeetCUDA/kernels/dot-product/dot_product.cu
284 template <const int NUM_THREADS = 256>
285 __global__ void dot(float *a, float *b, float *y, int N) {
286     int tid = threadIdx.x;
287     int idx = blockIdx.x * NUM_THREADS + tid;
288     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
289     __shared__ float reduce_smem[NUM_WARPS];
290
291     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
292     int warp = tid / WARP_SIZE;
293     int lane = tid % WARP_SIZE;
294
295     prod = warp_reduce_sum<WARP_SIZE>(prod);
296     if (lane == 0)
297         reduce_smem[warp] = prod;
298     __syncthreads();
299
300     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
301     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
302         prod = warp_reduce_sum<NUM_WARPS>(prod);
303     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
304         atomicAdd(y, prod);
305 }
306
307 // Dot Product + float4
308 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
309 // Block: (64, 1, 1), 256/4=64
310 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
311 // source: LeetCUDA/kernels/dot-product/dot_product.cu
312 template <const int NUM_THREADS = 256 / 4>
313 __global__ void dot_vec4(float *a, float *b, float *y, int N) {

```

```

314 int tid = threadIdx.x;
315 int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
316 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
317 __shared__ float reduce_smem[NUM_WARPS];
318
319 float4 reg_a = FLOAT4(a[idx]);
320 float4 reg_b = FLOAT4(b[idx]);
321 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
322                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
323                  : 0.0f;
324
325 int warp = tid / WARP_SIZE;
326 int lane = tid % WARP_SIZE;
327
328 prod = warp_reduce_sum<WARP_SIZE>(prod);
329 if (lane == 0)
330     reduce_smem[warp] = prod;
331 __syncthreads();
332
333 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
334 if (warp == 0)
335     prod = warp_reduce_sum<NUM_WARPS>(prod);
336 if (tid == 0)
337     atomicAdd(y, prod);
338 }
339
340 // =====
341 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
342 // =====
343 // 面试要点:
344 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
345 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
346 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
347
348 // ---- ReLU: y = max(0, x) ----
349 // Grid: ((N + 255) / 256, 1, 1)
350 // Block: (256, 1, 1)
351 // source: LeetCUDA/kernels/relu/relu.cu
352 __global__ void relu(float *x, float *y, int N) {
353     int idx = blockIdx.x * blockDim.x + threadIdx.x;
354     if (idx < N)
355         y[idx] = fmaxf(0.0f, x[idx]);
356 }
357
358 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
359 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (64, 1, 1)
362 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
363 // source: LeetCUDA/kernels/relu/relu.cu
364 __global__ void relu_vec4(float *x, float *y, int N) {
365     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
366     if (idx < N) {
367         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
368         float4 reg_y;
369         reg_y.x = fmaxf(0.0f, reg_x.x);
370         reg_y.y = fmaxf(0.0f, reg_x.y);
371         reg_y.z = fmaxf(0.0f, reg_x.z);
372         reg_y.w = fmaxf(0.0f, reg_x.w);
373         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
374     }
375 }
376

```

```

377 // ---- Elementwise Add: c = a + b ----
378 // Grid: ((N + 255) / 256, 1, 1)
379 // Block: (256, 1, 1)
380 // source: LeetCUDA/kernels/elementwise/elementwise.cu
381 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
382     int idx = blockIdx.x * blockDim.x + threadIdx.x;
383     if (idx < N)
384         c[idx] = a[idx] + b[idx];
385 }
386
387 // Elementwise Add + float4 向量化
388 // Grid: ((N + 255) / 256, 1, 1)
389 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
390 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
393     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
394     if ((idx + 3) < N) {
395         float4 reg_a = FLOAT4(a[idx]);
396         float4 reg_b = FLOAT4(b[idx]);
397         float4 reg_c;
398         reg_c.x = reg_a.x + reg_b.x;
399         reg_c.y = reg_a.y + reg_b.y;
400         reg_c.z = reg_a.z + reg_b.z;
401         reg_c.w = reg_a.w + reg_b.w;
402         FLOAT4(c[idx]) = reg_c;
403     } else if (idx < N) {
404         for (int i = 0; (idx + i) < N; i++) {
405             c[idx + i] = a[idx + i] + b[idx + i];
406         }
407     }
408 }
409
410 // ---- Histogram: y[a[i]]++ ----
411 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
412 // Grid: ((N + 255) / 256, 1, 1)
413 // Block: (256, 1, 1)
414 // source: LeetCUDA/kernels/histogram/histogram.cu
415 __global__ void histogram(int *a, int *y, int N) {
416     int idx = blockIdx.x * blockDim.x + threadIdx.x;
417     if (idx < N)
418         atomicAdd(&(y[a[idx]]), 1);
419 }
420
421 // =====
422 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
423 // =====
424 // 面试要点:
425 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
426 //     online (1-pass, 增量更新)
427 //   - Online Softmax 是 FlashAttention 的数学基础
428 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
429 //     + variance)
430 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
431
432 // =====
433 // Phase 3a: Softmax — 三级递进 (面试核心考点)
434 // =====
435 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
436 // 回答线索: naive(溢出) → safe → online
437
438 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
439 // grid(S*h/h, h), block(h), 一个 block 处理一个 token

```



```

440 // 问题: x 值很大时 exp(x) 溢出为 inf
441 template <const int NUM_THREADS = 256>
442 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
443 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
444 // source: LeetCUDA/kernels/softmax/softmax.cu
445 __global__ void softmax_per_token(float *x, float *y, int N) {
446     const int tid = threadIdx.x;
447     const int idx = blockIdx.x * blockDim.x + tid;
448
449     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
450     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
451     if (idx < N)
452         y[idx] = exp_val / exp_sum;
453 }
454
455 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
456 // 面试重点: 为什么 Softmax 需要 Safe?
457 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
458 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
459 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
460 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
461 // Grid: (S, 1, 1)
462 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
463 // source: LeetCUDA/kernels/softmax/softmax.cu
464 template <const int NUM_THREADS = 256>
465 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
466     const int tid = threadIdx.x;
467     const int idx = blockIdx.x * blockDim.x + tid;
468
469     // Pass 1: block reduce max — 找最大值
470     float val = (idx < N) ? x[idx] : -FLT_MAX;
471     float max_val = block_reduce_max<NUM_THREADS>(val);
472
473     // Pass 2: exp(x - max) → block reduce sum
474     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
475     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
476
477     if (idx < N)
478         y[idx] = exp_val / exp_sum;
479 }
480
481 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
482 // 面试重点: Online Softmax 为什么重要?
483 //   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
484 //   - Online Softmax 用递推公式合并为 1 遍遍历
485 //   - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
486 //   - 正是 FlashAttention 中 "online rescaling":
487 //       O_new = diag(exp(m_old - m_new)) * O_old + exp(m_cur - m_new) * P@V
488 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
489
490 // ---- Online Softmax 辅助结构 ----
491 // MD struct: 存储 running max (m) 和 running denominator (d = Σexp(x - m))
492 //
493 // 两种使用场景 (公式不同, 但结果等价):
494 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
495 //       m_new = max(m_old, x_i)
496 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
497 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
498 //       m = max(m1, m2)
499 //       d = d1*exp(m1-m) + d2*exp(m2-m)
500 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
501 //
502 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)

```

```

503 struct __align__(8) MD {
504     float m; // running max
505     float d; // running denominator (sum of exp(x - max))
506 };
507
508 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
509 //
510 // 不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
511 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
512 // (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
513 // (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
514 //
515 // 合并公式 (对称, 不依赖"新/旧"概念) :
516 // m = max(m1, m2)
517 // d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
518 //
519 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
520 template <const int kWarpSize = WARP_SIZE>
521 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
522 #pragma unroll
523     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
524         MD other;
525         other.m = __shfl_xor_sync(0xffffffff, value.m, mask, kWarpSize);
526         other.d = __shfl_xor_sync(0xffffffff, value.d, mask, kWarpSize);
527
528         bool value_bigger = (value.m > other.m);
529         MD bigger_m = value_bigger ? value : other;
530         MD smaller_m = value_bigger ? other : value;
531
532         // d_bigger 无需 rescale (其基准 max 已是全局 max) ,
533         // d_smaller 需 rescale: exp(m_smaller - m_bigger)
534         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
535         value.m = bigger_m.m;
536     }
537     return value;
538 }
539
540 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
541 // Grid: (S, 1, 1)
542 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
543 // source: LeetCUDA/kernels/softmax/softmax.cu
544 template <const int NUM_THREADS = 256>
545 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
546     int local_tid = threadIdx.x;
547     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
548     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
549     int warp_id = local_tid / WARP_SIZE;
550     int lane_id = local_tid % WARP_SIZE;
551
552     // 初始化: 每个线程持有一个 (max, denom) 对
553     MD val;
554     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
555     val.d = global_tid < N ? 1.0f : 0.0f;
556
557     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
558     __shared__ MD shared[WARP_NUM];
559     MD res = warp_reduce_md_op<WARP_SIZE>(val);
560
561     if (lane_id == 0)
562         shared[warp_id] = res;
563     __syncthreads();
564
565     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)

```

```

566 // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
567 if (local_tid < WARP_SIZE) {
568     MD block_res =
569         local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
570     block_res = warp_reduce_md_op<WARP_NUM>(block_res);
571     if (local_tid == 0) {
572         shared[0] = block_res;
573     }
574 }
575 __syncthreads();
576
577 // 用全局 max 和 denom 做最终 softmax
578 MD final_res = shared[0];
579 float d_total_inverse = __fdividef(1.0f, final_res.d);
580 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
581 if (global_tid < N) {
582     y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
583 }
584 }
585
586 // =====
587 // Phase 3b: RMS Normalization (1-pass reduce)
588 // =====
589 // 面试要点:
590 // - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
591 // - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
592 // - Llama 系列使用 RMS Norm
593 // - grid(N, K/K), block(K): 一行一个 block
594 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
595 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
596 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
597 template <const int NUM_THREADS = 128>
598 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
599     int tid = threadIdx.x;
600     int bid = blockIdx.x;
601     int idx = bid * blockDim.x + threadIdx.x;
602     const float epsilon = 1e-5f;
603
604     __shared__ float s_variance;
605     float value = (idx < N * K) ? x[idx] : 0.0f;
606     float variance = value * value;
607     variance = block_reduce_sum<NUM_THREADS>(variance);
608     if (tid == 0)
609         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
610     __syncthreads();
611     if (idx < N * K)
612         y[idx] = (value * s_variance) * g;
613 }
614
615 // RMS Norm + float4
616 // Grid: (N, 1, 1)
617 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
618 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
619 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
620 template <const int NUM_THREADS = 128 / 4>
621 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
622     int tid = threadIdx.x;
623     int bid = blockIdx.x;
624     int idx = (bid * blockDim.x + threadIdx.x) * 4;
625     const float epsilon = 1e-5f;
626
627     __shared__ float s_variance;
628     float4 reg_x = FLOAT4(x[idx]);

```

```

629 float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
630                                   reg_x.z * reg_x.z + reg_x.w * reg_x.w)
631                                   : 0.0f;
632 variance = block_reduce_sum<NUM_THREADS>(variance);
633 if (tid == 0)
634     s_variance = rsqrtf(variance / (float)K + epsilon);
635 __syncthreads();
636 float4 reg_y;
637 reg_y.x = reg_x.x * s_variance * g;
638 reg_y.y = reg_x.y * s_variance * g;
639 reg_y.z = reg_x.z * s_variance * g;
640 reg_y.w = reg_x.w * s_variance * g;
641 if (idx < N * K)
642     FLOAT4(y[idx]) = reg_y;
643 }
644
645 // =====
646 // Phase 3c: Layer Normalization (2-pass reduce)
647 // =====
648 // 面试要点:
649 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
650 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
651 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
652 // Grid: (N, 1, 1), 一行一个 block
653 // Block: (128, 1, 1), NUM_THREADS=K=128
654 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
655 template <const int NUM_THREADS = 128>
656 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
657     int tid = threadIdx.x;
658     int bid = blockIdx.x;
659     int idx = bid * blockDim.x + threadIdx.x;
660     const float epsilon = 1e-5f;
661
662     __shared__ float s_mean;
663     __shared__ float s_variance;
664     float value = (idx < N * K) ? x[idx] : 0.0f;
665
666     // Pass 1: compute mean
667     float sum = block_reduce_sum<NUM_THREADS>(value);
668     if (tid == 0)
669         s_mean = sum / (float)K;
670     __syncthreads(); // 必须等待 s_mean 对所有线程可见
671
672     // Pass 2: compute variance = (x - mean)2
673     float variance = (value - s_mean) * (value - s_mean);
674     variance = block_reduce_sum<NUM_THREADS>(variance);
675     if (tid == 0)
676         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
677     __syncthreads(); // 必须等待 s_variance 对所有线程可见
678
679     if (idx < N * K)
680         y[idx] = ((value - s_mean) * s_variance) * g + b;
681 }
682
683 // Layer Norm + float4
684 // Grid: (N, 1, 1)
685 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
686 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
687 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
688 template <const int NUM_THREADS = 128 / 4>
689 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
690                                 int K) {
691     int tid = threadIdx.x;

```

```

692 int bid = blockIdx.x;
693 int idx = (bid * blockDim.x + threadIdx.x) * 4;
694 const float epsilon = 1e-5f;
695
696 __shared__ float s_mean;
697 __shared__ float s_variance;
698 float4 reg_x = FLOAT4(x[idx]);
699 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
700
701 float sum = block_reduce_sum<NUM_THREADS>(value);
702 if (tid == 0)
703     s_mean = sum / (float)K;
704 __syncthreads();
705
706 float4 reg_x_hat;
707 reg_x_hat.x = reg_x.x - s_mean;
708 reg_x_hat.y = reg_x.y - s_mean;
709 reg_x_hat.z = reg_x.z - s_mean;
710 reg_x_hat.w = reg_x.w - s_mean;
711 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
712                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
713 variance = block_reduce_sum<NUM_THREADS>(variance);
714 if (tid == 0)
715     s_variance = rsqrtf(variance / (float)K + epsilon);
716 __syncthreads();
717
718 float4 reg_y;
719 reg_y.x = reg_x_hat.x * s_variance * g + b;
720 reg_y.y = reg_x_hat.y * s_variance * g + b;
721 reg_y.z = reg_x_hat.z * s_variance * g + b;
722 reg_y.w = reg_x_hat.w * s_variance * g + b;
723 if (idx < N * K)
724     FLOAT4(y[idx]) = reg_y;
725 }
726
727 // =====
728 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
729 // =====
730 // 面试要点:
731 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
732 //     [x1']   [cos(θ)  -sin(θ)] [x1]
733 //     [x2'] = [sin(θ)   cos(θ)] [x2]
734 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
735 //   - token_pos = idx / N: token 在序列中的位置
736 //   - token_idx = idx % N: token 内的维度对索引
737 //   - 输入 [seq_len, hidden_size], 输出同形状
738
739 // Grid: ((seq_len * N + 255) / 256, 1, 1)
740 // Block: (256, 1, 1)
741 // source: LeetCUDA/kernels/rope/rope.cu
742 __global__ void rope(float *x, float *out, int seq_len, int N) {
743     int idx = blockIdx.x * blockDim.x + threadIdx.x;
744     float x1 = x[idx * 2];
745     float x2 = x[idx * 2 + 1];
746     int token_pos = idx / N; // 序列位置
747     int token_idx = idx % N; // 维度对索引
748
749     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
750     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
751     float sin_v = sinf(token_pos * exp_v);
752     float cos_v = cosf(token_pos * exp_v);
753
754     // 2D 旋转

```

```

755 float out1 = x1 * cos_v - x2 * sin_v;
756 float out2 = x1 * sin_v + x2 * cos_v;
757 out[idx * 2] = out1;
758 out[idx * 2 + 1] = out2;
759 }
760
761 // =====
762 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
763 // =====
764 // 面试要点 (Bank Conflict 专题) :
765 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
766 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
767 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
768 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
769 //
770 // 转置的四步演进:
771 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
772 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
773 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
774 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
775 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
776
777 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写) ----
778 // 每个线程处理 1 个元素, block(16,16)
779 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
780 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
781 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
782 // Block: (16, 16, 1)
783 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
784 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
785     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
786     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
787     if (global_y < row && global_x < col) {
788         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
789         // 写入: y[global_y][global_x] = y[global_y * row +
790         // global_x], 列优先, 非合并
791         y[global_y * row + global_x] = x[global_x * col + global_y];
792     }
793 }
794
795 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write) ----
796
797 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
798 // Block: (16, 16, 1)
799 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
800 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
801 __global__ void mat_transpose_padded(
802     float *x, float *y, const int row, const int col) {
803     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
804     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
805     const int local_x = threadIdx.x;
806     const int local_y = threadIdx.y;
807
808     constexpr int WARP_SIZE_S = 16;
809     constexpr int PAD = 1;
810     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
811     __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
812
813     if (global_y * 4 < row && global_x < col) {
814         // Step 1: 从 global memory 读取 4 行, 写入 shared
815         // memory (行优先, 合并读取)
816         float4 x_val;
817         x_val.x = x[(global_y * 4) * col + global_x];

```

```

818     x_val.y = x[(global_y * 4 + 1) * col + global_x];
819     x_val.z = x[(global_y * 4 + 2) * col + global_x];
820     x_val.w = x[(global_y * 4 + 3) * col + global_x];
821     tile[local_y * 4][local_x] = x_val.x;
822     tile[local_y * 4 + 1][local_x] = x_val.y;
823     tile[local_y * 4 + 2][local_x] = x_val.z;
824     tile[local_y * 4 + 3][local_x] = x_val.w;
825     __syncthreads();
826
827     // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
828     float4 smem_val;
829     smem_val.x = tile[local_x * 4][local_y];
830     smem_val.y = tile[local_x * 4 + 1][local_y];
831     smem_val.z = tile[local_x * 4 + 2][local_y];
832     smem_val.w = tile[local_x * 4 + 3][local_y];
833
834     const int gid_x = blockIdx.x * blockDim.x;
835     const int gid_y = blockIdx.y * blockDim.y * 4;
836     const int out_y = gid_y + local_x * 4;
837     const int out_x = gid_x + local_y;
838
839     // 逐元素写回 (避免 reinterpret_cast<float4> 的对齐要求)
840     y[out_x * row + out_y + 0] = smem_val.x;
841     y[out_x * row + out_y + 1] = smem_val.y;
842     y[out_x * row + out_y + 2] = smem_val.z;
843     y[out_x * row + out_y + 3] = smem_val.w;
844 }
845 }
846
847 // =====
848 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
849 // =====
850 // 面试要点:
851 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
852 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
853 //   - 不同 K 值对应不同分块策略:
854 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
855 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
856 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
857 //
858 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
859
860 // ---- SGEMV K32: 基础 warp-per-row ----
861 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
862 // grid(M/4), 每个 warp 负责一行
863 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
864 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
865 // Block: (32, 4, 1), 每行 1 warp
866 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
867 // source: LeetCode/kernels/sgemv/sgemv.cu
868 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
869     int tx = threadIdx.x; // 0~31
870     int ty = threadIdx.y; // 0~3
871     int bx = blockIdx.x;
872     int lane = tx % WARP_SIZE; // 0~31
873     int m = bx * blockDim.y + ty; // 全局行号
874     if (m < M) {
875         float sum = 0.0f;
876         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
877         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
878         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
879 #pragma unroll
880         for (int w = 0; w < NUM_ITERS; ++w) {

```



```

881 // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
882 int k = w * WARP_SIZE + lane;
883 sum += a[m * K + k] * x[k];
884 }
885 sum = warp_reduce_sum<WARP_SIZE>(sum);
886 // 每个 warp 处理一行, lane 0 写回结果
887 if (lane == 0)
888     y[m] = sum;
889 }
890 }
891
892 // ---- SGEMV K128: float4 向量化 ----
893 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
894 // Grid: ((M + 3) / 4, 1, 1)
895 // Block: (32, 4, 1)
896 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
897 // source: LeetCUDA/kernels/sgemv/sgemv.cu
898 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
899     int tx = threadIdx.x; // 0~31
900     int ty = threadIdx.y; // 0~3
901     int bx = blockIdx.x;
902     int lane = tx % WARP_SIZE; // 0~31
903     int m = blockDim.y * bx + ty;
904
905     if (m < M) {
906         float sum = 0.0f;
907         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
908         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
909 #pragma unroll
910         for (int w = 0; w < NUM_ITERS; ++w) {
911             int k = (w * WARP_SIZE + lane) * 4;
912             float4 reg_x = FLOAT4(x[k]);
913             float4 reg_a = FLOAT4(a[m * K + k]);
914             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
915                 reg_a.w * reg_x.w);
916         }
917         sum = warp_reduce_sum<WARP_SIZE>(sum);
918         if (lane == 0)
919             y[m] = sum;
920     }
921 }
922
923 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
924 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
925 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
926 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
927 // Block: (32, 4, 1)
928 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
929 // source: LeetCUDA/kernels/sgemv/sgemv.cu
930 template <const int ROW_PER_WARP = 2>
931 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
932     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
933     int tx = threadIdx.x;
934     int ty = threadIdx.y;
935     int bx = blockIdx.x;
936     int lane = tx % WARP_SIZE;
937     int k = lane % K_WARP_SIZE; // 0~15
938     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
939     if (m < M) {
940         float sum = A[m * K + k] * x[k];
941         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
942         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
943         // 注意: 判断条件是 k == 0, 不是 lane == 0!

```



```

944     if (k == 0)
945         y[m] = sum;
946     }
947 }
948
949 // =====
950 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
951 // =====
952 // 面试要点 (GEMM 优化五层金字塔):
953 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
954 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
955 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
956 //   load/store 指令数 Level 4 — Tensor Core (MMA
957 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
958 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
959 //
960 // 计算密度递进:
961 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
962 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
963 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
964
965 // =====
966 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
967 // =====
968
969 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
970 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
971 // C = A x B, C[M, N] = A[M, K] x B[K, N]
972 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
973 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
974 // Block: (32, 32, 1), 1024 线程
975 // source: LeetCUDA/kernels/sgemm/sgemm.cu
976 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
977     constexpr int BM = 32; // vec 版: 32x4 = 128
978     constexpr int BN = 32; // vec 版: 32x4 = 128
979     constexpr int BK = 32;
980     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
981
982     int bx = blockIdx.x;
983     int by = blockIdx.y;
984     int tx = threadIdx.x;
985     int tid = threadIdx.y * blockDim.x + tx;
986
987     // 线程到 smem 的映射: 32×32 线程, 每个线程加载 a 和 b 各 1 个元素
988     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
989     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col
990     0~31, 每个线程加载 4 个元素, 8x4 = 32
991     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
992     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每
993     个线程加载 4 个元素, 32x4 = 128
994     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续)
995     [128x128]
996     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
997
998     float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32
999     x4, 32x4]=[128x128]大小
1000     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1001         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1002         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1003         // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
1004         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1005         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1006         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;

```

```

1003 // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1004 s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1005 __syncthreads(); // 确保整个 smem tile 加载完毕
1006
1007 #pragma unroll
1008 for (int k = 0; k < BK; ++k) {
1009     int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1010     int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1011     sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1012 }
1013 __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1014 }
1015 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1016 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1017 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1018 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1019 }
1020
1021 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1022 // 在 Level 1 基础上引入两层优化:
1023 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1024 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1025 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1026 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1027 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1028 // 1024 x 16 = 16384 = 128 x 128 ✓
1029 //
1030 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1031 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1032 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1033 //
1034 // 加载映射 (每线程 4 个元素, float4):
1035 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1036 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1037 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1038 //
1039 // △ Bank Conflict 提示 (面试加分点):
1040 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1041 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1042 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1043 //
1044 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1045 // Block: (32, 32, 1), 1024 线程
1046 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1047 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1048 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1049     constexpr int BM = 128;
1050     constexpr int BN = 128;
1051     constexpr int BK = 32;
1052     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1053     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1054
1055     int bx = blockIdx.x;
1056     int by = blockIdx.y;
1057     int tx = threadIdx.x;
1058     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1059
1060     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1061     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1062     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1063     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1064     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1065     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124

```

```

1066
1067 int load_gmem_a_m = by * BM + load_smem_a_m;
1068 int load_gmem_b_n = bx * BN + load_smem_b_n;
1069
1070 // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1071 int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1072 int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1073
1074 float sum[4][4] = {0.f};
1075 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1076     int load_gmem_a_k = bk * BK + load_smem_a_k;
1077     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1078     FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1079     int load_gmem_b_k = bk * BK + load_smem_b_k;
1080     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1081     FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1082     __syncthreads();
1083
1084 #pragma unroll
1085     for (int k = 0; k < BK; ++k) {
1086         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1087         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1088                             s_a[comp_smem_a_m_base + 1][k],
1089                             s_a[comp_smem_a_m_base + 2][k],
1090                             s_a[comp_smem_a_m_base + 3][k]};
1091         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1092                             s_b[k][comp_smem_b_n_base + 1],
1093                             s_b[k][comp_smem_b_n_base + 2],
1094                             s_b[k][comp_smem_b_n_base + 3]};
1095
1096 #pragma unroll
1097         for (int i = 0; i < 4; ++i) {
1098             #pragma unroll
1099             for (int j = 0; j < 4; ++j) {
1100                 sum[i][j] += a_vals[i] * b_vals[j];
1101             }
1102         }
1103         __syncthreads();
1104     }
1105
1106     // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1107     int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1108     int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1109     #pragma unroll
1110     for (int i = 0; i < 4; ++i) {
1111         int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1112         float4 reg_c;
1113         reg_c.x = sum[i][0];
1114         reg_c.y = sum[i][1];
1115         reg_c.z = sum[i][2];
1116         reg_c.w = sum[i][3];
1117         FLOAT4(c[store_gmem_c_addr]) = reg_c;
1118     }
1119 }
1120
1121 // =====
1122 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1123 // =====
1124 // 面试要点:
1125 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1126 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1127 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1128 // 寄存器)

```

```

1129 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1130 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1131 //
1132 // ★ TN 布局详解 (面试高频考点):
1133 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1134 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1135 // N = Normal (列优先, BLAS 原生格式)
1136 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1137 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1138 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1139 // BLAS=Normal) 即: C = op(A) × op(B) = AT × B? 不对! 在 row-major
1140 // 视角下: TN = A row-major [M×K], BT row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1141 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1142 // T on A: BLAS 把 row-major 的 A 视为 AT, 传 T 表示"转置回去"
1143 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1144 //
1145 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1146 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1147 // N = col-major (列优先, BLAS native = normal)
1148 //
1149 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1150 // - A: row-major [M, K], B: row-major [K, N]
1151 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1152 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1153 //
1154 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 BT=[N×K] row-major 存储 (A 行优先, B 列优先)
1155 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1156 // - BT [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1157 // - 优势: BT 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1158 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1159 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1160 //
1161 // WGMMA (Warp Group MMA, Hopper+):
1162 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1163 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1164 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1165 // threads) 做计算
1166 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1167 //
1168 // =====
1169 // Phase 5b-1: MMA PTX 宏定义
1170 // =====
1171 //
1172 // ---- gmem → smem: cp.async ----
1173 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1174 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1175 #define CP_ASYNC_WAIT_GROUP(n) \
1176     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1177 //
1178 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1179 #define CP_ASYNC_CG(dst, src, bytes) \
1180     asm volatile( \
1181         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1182         "l"(src), "n"(bytes))
1183 //
1184 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1185 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1186 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1187 // aligned: 要求 128-bit 对齐
1188 // trans: 转置加载 (用于 col-major 的 B 矩阵)
1189 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1190     asm volatile( \
1191         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \

```

```

1192 : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1193 : "r"(addr)) \
1194
1195 #define LDMATRIX_X2(R0, R1, addr) \
1196 asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1197 : "=r"(R0), "=r"(R1) \
1198 : "r"(addr)) \
1199
1200 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1201 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1202 #define LDMATRIX_X2_T(R0, R1, addr) \
1203 asm volatile( \
1204 "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1205 : "=r"(R0), "=r"(R1) \
1206 : "r"(addr)) \
1207
1208 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1209 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1210 // row.col: A 是 row-major, B 是 col-major
1211 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1212 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1213 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1214 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1215 asm volatile( \
1216 "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1217 "%4, %5}, {%6, %7}, {%8, %9};\n" \
1218 : "=r"(RD0), "=r"(RD1) \
1219 : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1220 "r"(RC1)) \
1221
1222 // ---- MMA 辅助函数 ----
1223 // div_ceil: 整数除法向上取整
1224 #define HOST_DEVICE_INLINE __device__ __host__ inline
1225 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1226     return (a % b != 0) ? (a / b + 1) : (a / b);
1227 }
1228
1229 // =====
1230 // Phase 5b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局
1231 // =====
1232 // 面试重点 — Tile Hierarchy:
1233 // MMA Atom: m16n8k16 (1 条 MMA 指令处理的最小 tile)
1234 // MMA Tile (warp): 2×4=8 个 MMA atom = [32×32] (一个 warp 的计算)
1235 // Warp Tile: 4×4=16 warps = [128×128] (整个 block 的计算)
1236 // 线程映射: 2×4=8 warps, 每个 warp 4×1=4 个 warp tile
1237 // 实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
1238 // → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1239 //
1240 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1241 // - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1242 // - B[K][N]: col-major (等价于 B^T[N][K] row-major), 全局索引 B[n*K+k] 即原 B 元素 (k,n)
1243 // (△ 内维连续的是 K) shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1244 // - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1245 // - ldmatrix B: 用 x2 (非转置), 因为 B^T 在 smem 中为 row-major, ldmatrix 逐行加载
1246 // B^T 的行即 B 的列, 天然匹配 MMA row.col 的 col-major B 输入, 无需 .trans
1247 // - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1248 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1249 // - grid.z = S 个 swizzle 分区
1250 // - grid.x = 每个分区内部需要发射多少个 128×128 的 N tiles
1251 // - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N 区域, 改善 L2 命中
1252 // Block: (256, 1, 1), 8 warps
1253 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1254 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,

```

```

1255     const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1256     const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1257     const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1258 __global__ void __launch_bounds__(256)
1259     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1260 // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1261 const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1262 const int by = blockIdx.y;
1263 constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1264 constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1265 constexpr int BK = MMA_K; // 16
1266
1267 // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1268 // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1269 // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1270 // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1271 // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1272 extern __shared__ half smem[];
1273 half *s_a = smem;
1274 half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1275 constexpr int s_a_stage_offset = BM * BK; // 128*16
1276 constexpr int s_b_stage_offset =
1277     BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major 的 smem 布局
1278
1279 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1280 const int warp_id = tid / WARP_SIZE; // 0~7
1281 const int lane_id = tid % WARP_SIZE; // 0~31
1282 const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1283 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1284
1285 // 线程到 global memory 的映射 (用于加载 A 和 B)
1286 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1287 int load_smem_a_m = tid / 2; // 0~127
1288 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1289 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1290 int load_smem_b_k =
1291     (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1292 int load_gmem_a_m = by * BM + load_smem_a_m;
1293 int load_gmem_b_n =
1294     bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏移
1295 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1296     return;
1297
1298 // 累加器: 每个 warp 计算 WARP_TILE_MxWARP_TILE_N=16 个 MMA tile
1299 uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1300
1301 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1302 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1303 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1304
1305 // 预加载前 (K_STAGE-1) 个 stage
1306 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [KxN])
1307 #pragma unroll
1308 for (int k = 0; k < (K_STAGE - 1); ++k) {
1309     int load_gmem_a_k = k * BK + load_smem_a_k;
1310     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1311     int load_gmem_b_k = k * BK + load_smem_b_k;
1312     int load_gmem_b_addr =
1313         load_gmem_b_n * K + load_gmem_b_k; // B^T: [n][k] row-major (即 B[k][n] col-major) △
1314
1315     uint32_t load_smem_a_ptr =
1316         (smem_a_base_ptr +
1317         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *

```



```

1318         sizeof(half));
1319 CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1320
1321 uint32_t load_smem_b_ptr =
1322     (smem_b_base_ptr +
1323      (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
1324       sizeof(half));
1325 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1326
1327 CP_ASYNC.COMMIT_GROUP();
1328 }
1329
1330 const int NUM_K_TILES = div_ceil(K, BK);
1331 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1332 __syncthreads();
1333
1334 // 主循环: K 维分块迭代
1335 #pragma unroll
1336 for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1337     // Stage 选择: 轮转方式 (round-robin)
1338     // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1339     // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
1340     int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1341     int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1342
1343     // 异步加载下一批数据到 smem_sel_next
1344     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用
1345     // n*K+k (row-major, 内维连续的是 K)
1346     int load_gmem_a_k = k * BK + load_smem_a_k;
1347     int load_gmem_a_addr =
1348         load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1349     int load_gmem_b_k = k * BK + load_smem_b_k;
1350     int load_gmem_b_addr =
1351         load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k], 内维连续的是 K
1352
1353     uint32_t load_smem_a_ptr =
1354         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1355         load_smem_a_m * BK + load_smem_a_k) *
1356          sizeof(half));
1357     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1358
1359     uint32_t load_smem_b_ptr =
1360         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1361         load_smem_b_n * BK + load_smem_b_k) *
1362          sizeof(half));
1363     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1364     CP_ASYNC.COMMIT_GROUP();
1365
1366     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1367     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1368     // B 用 x2 (非转置), smem 中 B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1369     uint32_t RA[WARP_TILE_M][4];
1370     uint32_t RB[WARP_TILE_N][2];
1371
1372     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1373     #pragma unroll
1374     for (int i = 0; i < WARP_TILE_M; ++i) {
1375         int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1376         int lane_smem_a_m =
1377             warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1378         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1379         uint32_t lane_smem_a_ptr =
1380             (smem_a_base_ptr +

```

```

1381         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1382         sizeof(half));
1383     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1384 }
1385
1386 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1387 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1388 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1389 #pragma unroll
1390 for (int j = 0; j < WARP_TILE_N; ++j) {
1391     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1392     int lane_smem_b_n =
1393         warp_smem_b_n + lane_id % 8;           // ldmatrix B 用 8 个 lane
1394     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1395     uint32_t lane_smem_b_ptr =
1396         (smem_b_base_ptr +
1397          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1398           sizeof(half));
1399     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1400 }
1401
1402 // MMA compute: 发射 WARP_TILE_M × WARP_TILE_N 条 MMA 指令
1403 #pragma unroll
1404 for (int i = 0; i < WARP_TILE_M; ++i) {
1405     #pragma unroll
1406     for (int j = 0; j < WARP_TILE_N; ++j) {
1407         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1408                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1409     }
1410 }
1411
1412 // 等待当前 stage 的异步加载完成, 然后 sync
1413 CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1414 __syncthreads();
1415 }
1416
1417 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1418 if ((K_STAGE - 2) > 0) {
1419     CP_ASYNC_WAIT_GROUP(0);
1420     __syncthreads();
1421 }
1422
1423 {
1424     #pragma unroll
1425     for (int k = 0; k < (K_STAGE - 1); k++) {
1426         uint32_t RA[WARP_TILE_M][4];
1427         uint32_t RB[WARP_TILE_N][2];
1428         int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);
1429
1430         #pragma unroll
1431         for (int i = 0; i < WARP_TILE_M; ++i) {
1432             int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1433             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1434             int lane_smem_a_k = (lane_id / 16) * 8;
1435             uint32_t lane_smem_a_ptr =
1436                 (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1437                                     lane_smem_a_m * BK + lane_smem_a_k) *
1438                  sizeof(half));
1439             LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1440         }
1441         #pragma unroll
1442         for (int j = 0; j < WARP_TILE_N; ++j) {
1443             int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;

```



```

1444     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1445     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1446     uint32_t lane_smem_b_ptr =
1447         (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1448             lane_smem_b_n * BK + lane_smem_b_k) *
1449             sizeof(half));
1450     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1451 }
1452 #pragma unroll
1453 for (int i = 0; i < WARP_TILE_M; ++i) {
1454     #pragma unroll
1455     for (int j = 0; j < WARP_TILE_N; ++j) {
1456         MMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1457             RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1458     }
1459 }
1460 }
1461 }
1462
1463 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1464 {
1465     for (int i = 0; i < WARP_TILE_M; ++i) {
1466         uint32_t RC0[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1467         uint32_t RC1[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1468         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1469         // 对 m16n8k16 的 C fragment:
1470         //   - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1471         //   - lane+1 持有同一行里的 {c2,c3}
1472         //   - lane+2 持有同一行里的 {c4,c5}
1473         //   - lane+3 持有同一行里的 {c6,c7}
1474         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由 lane%4==0
1475         // 的那个线程做一次 128-bit store。
1476     #pragma unroll
1477     for (int j = 0; j < WARP_TILE_N; ++j) {
1478         RC0[j][0] = RC[i][j][0];
1479         RC1[j][0] = RC[i][j][1];
1480         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1481         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1482         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1483         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1484         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1485         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1486     }
1487     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1488     if (lane_id % 4 == 0) {
1489         int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1490         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1491     #pragma unroll
1492     for (int j = 0; j < WARP_TILE_N; ++j) {
1493         int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1494         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1495         int store_gmem_c_addr_0 =
1496             store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1497         int store_gmem_c_addr_1 =
1498             (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1499         // 128-bit store: 一次写入 8 个 half
1500         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1501             *reinterpret_cast<float4*>(&RC0[j][0]);
1502         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1503             *reinterpret_cast<float4*>(&RC1[j][0]);
1504     }
1505 }
1506 }

```

```

1507 }
1508 }
1509
1510 // =====
1511 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1512 // =====
1513 // 面试要点 (WGMMMA vs MMA 对比) :
1514 //   - MMA: warp 级 (32 threads) , 同步执行
1515 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
1516 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA 的 32
1517 //     倍)
1518 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1519 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier
1520 //     同步
1521 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1522
1523 #if defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900
1524 // ---- WGMMMA 辅助函数 ----
1525 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1526 #define WGMMMA_COMMIT_GROUP() \
1527     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1528 #define WGMMMA_WAIT_GROUP(n) \
1529     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1530
1531 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1532 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
1533 //   encoded = (x & 0x3FFFF) >> 4
1534 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1535 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1536 // 可省略以扩大可表示范围。
1537 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
1538 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1539
1540 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1541 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1542 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1543 //
1544 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
1545 //   -----
1546 //   | 位域          | 范围      | 大小 | 含义
1547 //   |-----|-----|-----|-----|
1548 //   | start_address  | [0, 14)   | 14   | smem 基址编码
1549 //   | (unused)       | [14, 16)  | 2    | -
1550 //   | leading_byte_offset | [16, 30) | 14   | 次要维度的字节步长编码
1551 //   | (unused)       | [30, 32)  | 2    | -
1552 //   | stride_byte_offset | [32, 46) | 14   | 主要维度的字节步长编码
1553 //   | (unused)       | [46, 49)  | 3    | -
1554 //   | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
1555 //   | (unused)       | [52, 62)  | 10   | -
1556 //   | layout_type    | [62, 64)  | 2    | swizzle 模式
1557 //   -----
1558 //   layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1559 //
1560 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1561 //   - 128B swizzle atom 在 128-bit 单位下为 8×8
1562 //   - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1563 //   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1564 //   - 这是 stride_byte_offset=1024 的来源
1565 //
1566 //   - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
1567 //     此处填 16 仅为占位, 编码后为 1。
1568 //
1569 //   - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。

```

```

1570 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1571 //
1572 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1573 __device__ inline uint64_t make_smem_desc(half *ptr) {
1574 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1575 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1576 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1577
1578 // 从零开始: 所有位域初始化为 0。
1579 uint64_t desc = 0x0000000000000000;
1580
1581 // — bits [0, 14): start_address —
1582 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit),
1583 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1584 // 可寻址范围:  $2^{(14+4)} = 2^{18} = 256K$  个 half = 512 KB 共享内存。
1585 desc |= SMEM_DESC_ENCODE(addr);
1586
1587 // — bits [16, 30): leading_byte_offset —
1588 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1589 // << 16 将编码值放到 bits [16, 30)。
1590 // K-Major + 128B swizzle 下该字段 unused (hardware assumes 1),
1591 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1592 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1593 // 对 INTERLEAVE: 同一 8x2 brick 内第一列到第二列的字节偏移。
1594 // 对 swizzle: unused, assumed to be 1。
1595 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1596
1597 // — bits [32, 46): stride_byte_offset —
1598 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1599 // << 32 将编码值放到 bits [32, 46)。
1600 // 1024 的来源 (K-Major + 128B swizzle + half):
1601 // 一个 128B swizzle atom 覆盖  $64(K) \times 8(M)$  个 half。
1602 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越  $8 \times 64 \times 2 = 1024$  bytes。
1603 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组)。
1604 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1605
1606 // — bits [62, 64): layout_type (swizzle mode) —
1607 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1608 // 1 = 128B swizzle mode。
1609 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1610 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1611 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1612 desc |= 1llu << 62;
1613
1614 return desc;
1615 }
1616
1617 // ---- WGMMA PTX 指令宏 ----
1618 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1619 // m64n128k16: M=64, N=128, K=16
1620 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1621 // 每线程 32 个 uint32 输出: D=64x128=8192 half, warpgroup 128 线程分担,
1622 // 每线程 64 half = 32 uint32
1623 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1624 // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1625 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1626                                     TransB) \
1627 { \
1628     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1629     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1630     asm volatile( \
1631         "{\n" \
1632         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \

```

```

1633     "{%0, %1, %2, %3, %4, %5, %6, %7, " \
1634     "%8, %9, %10, %11, %12, %13, %14, %15, " \
1635     "%16, %17, %18, %19, %20, %21, %22, %23, " \
1636     "%24, %25, %26, %27, %28, %29, %30, %31}," \
1637     "%32," \
1638     "%33," \
1639     "%34, %35, %36, %37, %38;\n" \
1640     "}\n" \
1641     : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1642     "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1643     "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1644     "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1645     "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1646     "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1647     "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1648     "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1649     : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
1650     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1651     "n"(int32_t(TransB))); \
1652 }
1653
1654 // ---- TMA Shared Memory Layout ----
1655 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1656 //
1657 // 每个 stage 包含两块 smem:
1658 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1659 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1660 // 注意 B 的 smem 布局是 row-major [BK, BN], 物理上按 K-major 排列 (imm-trans-b=0),
1661 // 与 A 的 K-major 配合供 WGMMMA 读取。128B swizzle 将 [BK,BN] row-major 重新映射为
1662 // K-major 布局, 使得 K 维元素在 128B atom 内连续, 区别于 MMA TN 布局下的 B^T[N,K]。
1663 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1664     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1665     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (K-major 布局, 供 WGMMMA imm-trans-b=0 直
        接读取)
1666 };
1667
1668 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1669 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
1670 //
1671 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1672 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1673 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1674 //
1675 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1676 //   将 A/B tile 从 HBM 搬到 shared memory。
1677 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
1678 //
1679 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1680 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪
1681 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可被覆盖
1682 //
1683 // 本节重点理解:
1684 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1685 //      即可搬运整个 2D tile, 无需所有线程参与。
1686 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1687 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1688 //      Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1689 //
1690 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1691 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1692 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
1693 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
1694 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块

```

```

1695 // Block Tile:      C[128,128] = A[128,64] × B[64,128]
1696 // Threads:        256 = 2 warpgroups × 128 threads/warpgroup
1697 // Producer(WG0):   128 threads (仅 thread 0 做 TMA 提交)
1698 // Consumer(WG1):   128 threads = 4 warps (全部参与 WGMMMA 和写回)
1699 //
1700 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步，确保 HBM→SMEM
1701 // 的延迟被计算完全掩盖。
1702 //
1703 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1704 // - grid.z = S 个 swizzle 分区，将连续 block 打散到不同 N 区域改善 L2 命中
1705 // Block: (256, 1, 1), 2 warpgroups
1706 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1707 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1708           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1709           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1710           const bool BLOCK_SWIZZLE = false>
1711 __global__ void __launch_bounds__(NUM_THREADS)
1712 hgemm_wgmma_stages_tn(
1713     int M, int N, int K, half *C,
1714     const CUtensorMap *__restrict__ tensorMapA,
1715     const CUtensorMap *__restrict__ tensorMapB) {
1716
1717     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1718     // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1719     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1720     // 不展开宿主侧 create_tensor_map 细节。
1721
1722     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1723     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1724     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1725     const int by = blockIdx.y;
1726     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1727     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1728     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1729     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1730     // 当只有一个 consumer 时, 它负责全部 BM=128 行
1731     constexpr int B_WG_M = BM / num_consumers; // 128
1732
1733     // 边界检查: 确保当前 block 不超出 M/N 范围
1734     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1735         return;
1736
1737     // ---- Shared Memory 分配 ----
1738     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1739     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1740     extern __shared__ __align__(128) uint8_t smem[];
1741     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1742         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem);
1743     half *s_a = s.A;
1744     half *s_b = s.B;
1745
1746     // ---- cuda::barrier 初始化 ----
1747     // 每个 stage 有两个 barrier:
1748     // full[qidx]: Producer thread 0 发 full 信号, 128 个 Consumer 线程等 full
1749     // empty[qidx]: Consumer 发 empty 信号, Producer thread 0 等 empty
1750     // 每轮参与 arrive 的总人数 = 128 (consumer) + 1 (producer) = 129
1751     __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1752     __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1753
1754     // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
1755     const int num_blocks_k = K / BK;
1756     const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1757     const int tid = threadIdx.x % 128; // 0~127 within warpgroup

```

```

1758 // 初始化 barriers (仅 thread 0 执行)
1759 if (threadIdx.x == 0) {
1760     for (int i = 0; i < K_STAGE; ++i) {
1761         // init 的第二个参数是 barrier 的 arrive count:
1762         // num_consumers * 128 + 1 = 1 * 128 + 1 = 129
1763         // 即每轮需要 128 个 consumer 线程 + 1 个 producer 线程都 arrive 后,
1764         // barrier 才翻转 phase 并唤醒等待线程。
1765         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1766         init(&empty[i], num_consumers * 128 + 1); // same
1767     }
1768     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1769     // 对 async proxy (TMA/WGMMA) 可见。
1770     cuda::device::fence_proxy_async_shared_cta();
1771 }
1772 __syncthreads();
1773
1774 // =====
1775 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1776 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
1777 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
1778 // =====
1779 if (wg_idx == 0) {
1780     if (tid == 0) {
1781         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1782         int qidx = 0;
1783         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1784              ++block_k_iter, ++qidx) {
1785             if (qidx == K_STAGE)
1786                 qidx = 0;
1787
1788             // Step P1: 等待 Consumer 释放此 stage (empty 信号)
1789             // empty[qidx].arrive() 表示 Producer 自己"已准备好等待",
1790             // 然后 wait() 阻塞直到 barrier phase 翻转 (即所有 Consumer 都已 arrive)。
1791             empty[qidx].wait(empty[qidx].arrive());
1792
1793             // Step P2: 提交 TMA 2D 拷贝指令
1794             // cp_async_bulk_tensor_2d_global_to_shared:
1795             // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1796             // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1797             // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1798             // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1799             //
1800             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1801             // 无需线程逐元素搬运, 零寄存器开销。
1802             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1803             cuda::device::cp_async_bulk_tensor_2d_global_to_shared(
1804                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1805                 full[qidx]);
1806
1807             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1808             cuda::device::cp_async_bulk_tensor_2d_global_to_shared(
1809                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1810                 full[qidx]);
1811
1812             // Step P3: 通知 Consumer 此 stage 已准备好 (full 信号)
1813             // barrier_arrive_tx 除了 arrive 之外还额外告知硬件:
1814             // 本次 TMA 事务预期传输的字节数。
1815             // Consumer 的 full[qidx].wait() 会等待这 (BK*BN+BK*BM)*sizeof(half)
1816             // 字节全部写入 smem 后才返回。
1817             cuda::device::barrier_arrive_tx(
1818                 full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1819         }
1820     }

```



```

1821 }
1822 }
1823 // =====
1824 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1825 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1826 // 所有 128 个线程 (4 warps) 全部参与。
1827 // =====
1828 else {
1829     // Step C0: Consumer 初始时"准备就绪"
1830     // 对所有 stage 的 empty barrier 执行 arrive, 表示初始时所有 stage
1831     // 都是"空的" (可被 Producer 写入)。
1832     // 如果没有这一步, Producer 的 empty[qidx].wait() 在第一轮会永远阻塞。
1833     for (int i = 0; i < K_STAGE; ++i) {
1834         empty[i].arrive();
1835     }
1836
1837     // 累加器寄存器声明
1838     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1839     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1840     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1841     //   - d[*][g][*]: N 方向第 g 组 16 列
1842     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
1843     // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1844     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1845     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1846
1847     int qidx = 0;
1848     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1849     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1850         ++block_k_iter, ++qidx) {
1851         if (qidx == K_STAGE)
1852             qidx = 0;
1853
1854         // Step C1: 等待 Producer 的 full 信号
1855         // 表示 stage qidx 的 A/B tile 数据已通过 TMA 搬运到 smem。
1856         full[qidx].wait(full[qidx].arrive());
1857
1858         // Step C2: 发射 WGMMMA 指令序列
1859         //
1860         // WGMMMA 指令序列的固定模式:
1861         //   WGMMMA_FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP
1862         //
1863         // WGMMMA_FENCE: 确保 smem 写可见、accum 寄存器准备好。
1864         //   是一条内存 fence 指令, 建立 async proxy 与 generic proxy 之间的
1865         //   可见性顺序。必须在发射 WGMMMA 之前执行。
1866         //
1867         // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1868         //   执行 D[64*128] += A[64*16] * B[16*128]
1869         //   其中 A 的 smem 指针由 descA 描述 (K-major), B 由 descB 描述 (K-major)
1870         //   imm-trans-a=0/imm-trans-b=0 表示 A/B 都是 K-major (非转置)。
1871         //   ScaleD=1: 累加模式 (不清零), 用于 K 维累加。
1872         //   ScaleA=ScaleB=1: 不翻转符号。
1873         WGMMMA_FENCE();
1874
1875         // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
1876         // 每个 atom 处理 64 行 M 方向数据。
1877 #pragma unroll
1878         for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1879             // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1880             // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1881             half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1882
1883             // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)

```



```

1884 // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
1885 #pragma unroll
1886 for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1887     // 第 k_it 次 WGMMMA:
1888     //   A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
1889     //   B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
1890     //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1891     //   第 k_it*16 行开始取 16 行，每行 BN=128 列。
1892     //   ScaleD=1: 累加 (K 维迭代需要累积结果)
1893     WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1894                                   s_b + qidx * BK * BN + k_it * WGMMMA_K,
1895                                   1, // ScaleD=1: accumulate (不清零)
1896                                   1, 1, 0, 0);
1897 }
1898 }
1899
1900 // Step C3: 提交并等待 WGMMMA 完成
1901 // COMMIT_GROUP: 将此前发射的所有 WGMMMA 指令归为一组提交。
1902 // WAIT_GROUP(0): 等待之前 commit 的所有 WGMMMA 完成 (0 表示等待所有组)。
1903 // 注意 WGMMMA 是异步执行 (async proxy)，这些指令用于同步 completion。
1904 WGMMMA_COMMIT_GROUP();
1905 WGMMMA_WAIT_GROUP(0);
1906
1907 // Step C4: 释放此 stage (发 empty 信号给 Producer)
1908 // 通知 Producer stage qidx 的 smem 已使用完毕，可以安全覆盖。
1909 empty[qidx].arrive();
1910 }
1911
1912 // =====
1913 // Epilogue: 将寄存器中的累加结果写回 global memory C
1914 // =====
1915 //
1916 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
1917 //
1918 // 对 warpgroup 内每个线程，输出 64 个 half = 32 个 uint32。
1919 // 这些 half 分布在 d[2][8][4] 数组中:
1920 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
1921 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
1922 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
1923 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
1924 //
1925 // 其中:
1926 //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
1927 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
1928 //
1929 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
1930 //   +-----+-----+
1931 //   | reg[0] | reg[2] | <- rows [row, row+8)
1932 //   |(col,+2)|(col+8)|
1933 //   +-----+-----+
1934 //   | reg[1] | reg[3] | <- rows [row+8, row+16)
1935 //   |(col,+2)|(col+8)|
1936 //   +-----+-----+
1937 //   每个 reg 包含 2 个连续 half (col 和 col+2)，用 uint32 存储。
1938 //
1939 // 三维遍历:
1940 //   m_it=0: rows 0~63, m_it=1: rows 64~127
1941 //   g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
1942 //   每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
1943 //   128 线程 * 128 half = 16384 half = 128 * 128 ok
1944
1945 const int lane = tid % 32;
1946 const int warp = tid / 32;

```

```

1947 // row: 当前线程在当前 WGMMMA atom 中负责的起始行
1948 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]
1949 const int row = warp * 16 + lane / 4;
1950 // block_C: 当前 C tile 在 global memory 中的起始地址
1951 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
1952 half *block_C = C + by * BM * N + bx * BN;
1953
1954 #pragma unroll
1955 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1956     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
1957     #pragma unroll
1958     for (int g = 0; g < WGMMMA_N / 16; ++g) {
1959         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
1960         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
1961         // 左上象限: (row, col)
1962         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1963             d[m_it][g][0];
1964         // 左下象限: (row+8, col)
1965         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1966             d[m_it][g][1];
1967         // 右上象限: (row, col+8)
1968         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1969             d[m_it][g][2];
1970         // 右下象限: (row+8, col+8)
1971         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1972             d[m_it][g][3];
1973     }
1974 }
1975 }
1976 }
1977 #endif /* __CUDA_ARCH__ >= 900 */
1978
1979 // =====
1980 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
1981 // =====
1982 // 面试要点 (FlashAttention 算法):
1983 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
1984 // 但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
1985 // 2. FA 三板斧:
1986 // a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
1987 // b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
1988 // c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
1989 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
1990 // - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
1991 // - 优点: 减少 warp 间通信和 shuffle
1992 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
1993 // for each K,V tile:
1994 //     S_cur = Q @ K^T // 未缩放, 存入 R_S
1995 //     m_new = max(m_old, row_max(S_cur * scale))
1996 //     P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
1997 //     l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
1998 //     O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
1999 //     O_final = O_new / l_final
2000 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2001 // - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
2002 // - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2003 // 后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2004 // - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
2005 // 都天然同构”的通用结论
2006 //
2007 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2008 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2009 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64

```

```

2010 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
2011 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2012
2013 // ---- 寄存器填充辅助函数 ----
2014 template <typename T, int M, const int N, const int K = 2>
2015 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2016 #pragma unroll
2017     for (int i = 0; i < M; ++i)
2018 #pragma unroll
2019         for (int j = 0; j < N; ++j)
2020 #pragma unroll
2021             for (int k = 0; k < K; ++k)
2022                 R[i][j][k] = val;
2023 }
2024
2025 template <typename T, int M, const int N = 2>
2026 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2027 #pragma unroll
2028     for (int i = 0; i < M; ++i)
2029 #pragma unroll
2030         for (int j = 0; j < N; ++j)
2031             R[i][j] = val;
2032 }
2033
2034 // =====
2035 // FlashAttention-2 Split-Q Kernel (完整实现)
2036 // =====
2037 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2038 //
2039 // Tile 设计 (以 kHeadDim=64 为例) :
2040 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2041 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2042 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2043 //
2044 // 执行流程:
2045 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2046 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2047 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2048 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
2049 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2050 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2051 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2052 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2053 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2054 //   8) 最终 rescale: O_final = (1/l_final)* O_final
2055 //   9) Epilogue: warp shuffle + 128-bit collective store
2056
2057 template <
2058     const int kHeadDim,           // head dim: 32, 64, 128
2059     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
2060     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
2061     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
2062     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
2063     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
2064     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2065     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
2066     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
2067     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
2068     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
2069     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2070     const int kStage,             // pipeline stages for K: 1 or 2
2071     const int kPad>
2072 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)

```

```

2073     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2074                                   int QKV_seqlen, int QKV_head) {
2075
2076     // Tile dimensions
2077     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2078     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2079     constexpr int kNumThreads =
2080         WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2081     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2082     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2083     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2084     const float scale = 1.0f / sqrtf((float)kHeadDim);
2085
2086     // Block indexing
2087     const int QKV_batch_id = blockIdx.y / QKV_head;
2088     const int QKV_head_id = blockIdx.y % QKV_head;
2089     const int Q_tile_id = blockIdx.x;
2090     const int tid = threadIdx.x;
2091     const int warp_id = tid / WARP_SIZE;
2092     const int lane_id = tid % WARP_SIZE;
2093     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2094     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2095
2096     // Global memory base offsets for this (batch, head)
2097     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2098     const int Q_gmem_offset =
2099         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2100         kHeadDim;
2101     const int K_gmem_offset = Q_gmem_offset;
2102     const int V_gmem_offset = Q_gmem_offset;
2103     const int O_gmem_offset = Q_gmem_offset;
2104
2105     // Thread-to-smem mapping for cooperative load
2106     int load_smem_Q_Br = tid / (kNumThreads / Br);
2107     int load_smem_Q_d =
2108         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2109     int load_smem_K_Bc = tid / (kNumThreads / Bc);
2110     int load_smem_K_d =
2111         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2112     int load_smem_V_Bc = tid / (kNumThreads / Bc);
2113     int load_smem_V_d =
2114         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2115
2116     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2117     if (load_gmem_Q_Br >= QKV_seqlen)
2118         return;
2119
2120     // ---- Shared memory layout ----
2121     extern __shared__ half smem[];
2122     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2123     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2124     half *Q_tile_smem = smem;
2125     half *K_tile_smem = Q_tile_smem + Q_tile_size;
2126     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2127     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2128     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2129
2130     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2131     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2132     uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2133
2134     // ---- Online Softmax persistent state ----
2135     // lane_block_row_max_old[i][r]: running max for row r of warp tile i

```

```

2136 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2137 float lane_block_row_max_old[kValTileSeqLenQ][2];
2138 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2139 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2140 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2141
2142 // ---- Register allocation ----
2143 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2144 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2145 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2146 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2147 // 对单个 m16n8k16 tile 而言:
2148 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
2149 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
2150 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2151 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
2152 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
2153 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2154           [2]; // O accumulator (final output)
2155
2156 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2157 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2158
2159 // =====
2160 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2161 // =====
2162 {
2163     int load_gmem_Q_addr =
2164         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2165     uint32_t load_smem_Q_ptr =
2166         smem_Q_base_ptr +
2167         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2168 #pragma unroll
2169     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2170         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2171     }
2172     CP_ASYNC_COMMIT_GROUP();
2173 }
2174
2175 // =====
2176 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2177 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
2178 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
2179 // =====
2180 if constexpr (kStage > 1) {
2181 #pragma unroll
2182     for (int stage = 0; stage < (kStage - 1); ++stage) {
2183         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2184         int load_gmem_K_addr =
2185             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2186         uint32_t load_smem_K_ptr =
2187             smem_K_base_ptr +
2188             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2189              load_smem_K_d) *
2190             sizeof(half);
2191 #pragma unroll
2192         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2193             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2194         }
2195         CP_ASYNC_COMMIT_GROUP();
2196     }
2197     CP_ASYNC_WAIT_GROUP(kStage - 2);
2198     __syncthreads();

```

```

2199 }
2200
2201 // =====
2202 // Step 3: 外循环 — 沿 K seqLen 迭代 (Tc = ceil(seqLen/Bc))
2203 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2204 // =====
2205 #pragma unroll 1
2206 for (int tile_K_seqLen = 0; tile_K_seqLen < Tc; ++tile_K_seqLen) {
2207     int smem_sel = tile_K_seqLen % kStage;
2208     int smem_sel_next = (tile_K_seqLen + (kStage - 1)) % kStage;
2209
2210     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2211     if constexpr (kStage > 1) {
2212         // 只有 kStage>1 才能真正做 K 的 pipeline:
2213         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2214         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
2215         // Load current V tile (no pipeline for V — one stage is enough)
2216         {
2217             int load_gmem_V_Bc = tile_K_seqLen * Bc + load_smem_V_Bc;
2218             int load_gmem_V_addr =
2219                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2220             uint32_t load_smem_V_ptr =
2221                 smem_V_base_ptr +
2222                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2223 #pragma unroll
2224             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2225                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2226             }
2227             CP_ASYNC_COMMIT_GROUP();
2228         }
2229
2230         // Prefetch next K tile (pipelined)
2231         if ((tile_K_seqLen + 1) < Tc) {
2232             int load_gmem_K_Bc = (tile_K_seqLen + 1) * Bc + load_smem_K_Bc;
2233             int load_gmem_K_addr =
2234                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2235             uint32_t load_smem_K_ptr =
2236                 smem_K_base_ptr +
2237                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2238                  load_smem_K_d) *
2239                 sizeof(half);
2240 #pragma unroll
2241             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2242                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2243             }
2244             CP_ASYNC_COMMIT_GROUP();
2245         }
2246     }
2247
2248     // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2249     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2250 #pragma unroll
2251     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2252         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2253 #pragma unroll
2254         for (int i = 0; i < kValTileSeqLenQ; ++i) {
2255             int warp_smem_Q_Br =
2256                 warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2257             int lane_smem_Q_Br =
2258                 warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2259             int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2260             uint32_t lane_smem_Q_ptr =
2261                 smem_Q_base_ptr +

```



```

2262         (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2263     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2264               lane_smem_Q_ptr);
2265 }
2266
2267 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2268 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2269 #pragma unroll
2270 for (int j = 0; j < kValTileSeqLenK; ++j) {
2271     int warp_smem_K_Bc =
2272         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2273     int lane_smem_K_Bc =
2274         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2275     int lane_smem_K_d =
2276         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2277     uint32_t lane_smem_K_ptr =
2278         smem_K_base_ptr +
2279         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2280          lane_smem_K_d) *
2281         sizeof(half);
2282     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2283 }
2284
2285 // MMA: S[tile] += Q[tile] @ K^T[tile]
2286 #pragma unroll
2287 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2288     #pragma unroll
2289     for (int j = 0; j < kValTileSeqLenK; ++j) {
2290         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2291                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2292                 R_S[i][j][1]);
2293     }
2294 }
2295 } // end loop over d
2296 __syncthreads();
2297
2298 // =====
2299 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to
2300 // R_S
2301 // =====
2302 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2303 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2304 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2305 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2306 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2307 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2308 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2309 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2310 // kValTileSeqLenK tiles.
2311
2312 float lane_row_max_new[kValTileSeqLenQ][2];
2313 float lane_row_sum_new[kValTileSeqLenQ][2];
2314 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2315 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2316
2317 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2318 #pragma unroll
2319 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2320     #pragma unroll
2321     for (int j = 0; j < kValTileSeqLenK; ++j) {
2322         // Extract half values from R_S registers (C matrix fragment layout)
2323         float2 t_reg_S_0 =
2324             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}

```



```

2325     float2 t_reg_S_1 =
2326         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2327     // S = (Q@K^T) * scale
2328     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2329     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2330     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2331     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2332 }
2333 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2334 // {0,4,8,...,28} hold valid data)
2335 lane_row_max_new[i][0] =
2336     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2337 lane_row_max_new[i][1] =
2338     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2339 }
2340
2341 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2342 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2343 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2344 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2345 #pragma unroll
2346 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2347     // m_new = max(m_old, m_cur)
2348     float block_row_max_new_0 =
2349         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2350     float block_row_max_new_1 =
2351         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2352
2353     #pragma unroll
2354     for (int j = 0; j < kValTileSeqLenK; ++j) {
2355         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2356         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2357
2358         // P = exp(S * scale - m_new), 用 fma 保证精度
2359         t_reg_S_0.x =
2360             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2361         t_reg_S_0.y =
2362             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2363         t_reg_S_1.x =
2364             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2365         t_reg_S_1.y =
2366             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2367
2368         // Accumulate row sums
2369         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2370         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2371
2372         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2373         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2374         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2375     }
2376
2377     // Warp-level reduce sum (warp_size = 4, same as max)
2378     lane_row_sum_new[i][0] =
2379         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2380     lane_row_sum_new[i][1] =
2381         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2382 }
2383 __syncthreads();
2384
2385 // =====
2386 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2387 // =====

```

```

2388 // Wait for V to be ready before computing P@V
2389 if constexpr (kStage > 1) {
2390     if ((tile_K_seqLen + 1) < Tc) {
2391         CP_ASYNC_WAIT_GROUP(1);
2392     } else {
2393         CP_ASYNC_WAIT_GROUP(0);
2394     }
2395 } else {
2396     CP_ASYNC_WAIT_GROUP(0);
2397 }
2398 __syncthreads();
2399
2400 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2401
2402 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2403 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2404 #pragma unroll
2405 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2406     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2407     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2408     // transposed ldmatrix
2409 #pragma unroll
2410 for (int j = 0; j < kValTileHeadDimV; ++j) {
2411     int warp_smem_V_d =
2412         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2413     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2414     int lane_smem_V_d = warp_smem_V_d;
2415     uint32_t lane_smem_V_ptr =
2416         smem_V_base_ptr +
2417         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2418     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2419 }
2420
2421 // P matrix layout in R_S[i][j][2]:
2422 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2423 //   | 64x64 | warp_KV 0 |
2424 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2425 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2426 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2427 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2428 // tile_V_Bc selects which 16-column slice of P to use:
2429 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2430 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2431 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2432 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2433 // 对应的 MMA A fragment 布局可以这样记:
2434 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2435 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2436 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2437 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2438 // fragment 都无条件成立的通用结论。layout转换逻辑:
2439 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2440 int w = tile_V_Bc * 2;
2441 #pragma unroll
2442 for (int i = 0; i < kValTileSeqLenP; ++i) {
2443 #pragma unroll
2444 for (int j = 0; j < kValTileHeadDimV; ++j) {
2445     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2446         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2447         R_V[j][0], R_V[j][1], // B fragment = V
2448         R_0[i][j][0], R_0[i][j][1]); // C fragment output
2449     }
2450 }

```

```

2451 } // end for tile_V_Bc
2452 __syncthreads();
2453
2454 // =====
2455 // 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$ 
2456 // =====
2457 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{\text{old}} - m_{\text{new}})$  做 0 与 1 的 rescale
2458 #pragma unroll
2459 for (int i = 0; i < kValTileSeqLenP; ++i) {
2460     float block_row_max_new_0 = lane_row_max_new[i][0];
2461     float block_row_max_new_1 = lane_row_max_new[i][1];
2462     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2463     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2464
2465     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2466     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2467
2468     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2469     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2470
2471     // Handle first iteration:  $m_{\text{old}} = -\text{inf}$ , need to use  $m_{\text{new}}$  directly
2472     block_row_max_old_0 =
2473         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2474     block_row_max_old_1 =
2475         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2476
2477     float rescale_o_factor_0 =
2478         __expf(block_row_max_old_0 - block_row_max_new_0);
2479     float rescale_o_factor_1 =
2480         __expf(block_row_max_old_1 - block_row_max_new_1);
2481
2482     // Rescale  $O_{\text{old}}$  + Add  $P@V$  in one fused step
2483     #pragma unroll
2484     for (int j = 0; j < kValTileHeadDimV; ++j) {
2485         //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
2486         //   reg[0] -> rows 0~7 的 {c0,c1}
2487         //   reg[1] -> rows 8~15 的 {c2,c3}
2488         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2489         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2490         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2491         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2492
2493         //  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  (fused multiply-add)
2494         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2495         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2496         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2497         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2498
2499         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2500         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2501     }
2502
2503     // Update l:  $l_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * l_{\text{old}} + \text{row\_sum}(P)$ 
2504     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2505     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2506     lane_block_row_sum_old[i][0] = __fmaf_rn(
2507         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2508     lane_block_row_sum_old[i][1] = __fmaf_rn(
2509         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2510
2511     // Update m
2512     lane_block_row_max_old[i][0] = block_row_max_new_0;
2513     lane_block_row_max_old[i][1] = block_row_max_new_1;

```

```

2514 }
2515
2516 // Wait for next K tile to be ready in smem before next iteration
2517 if constexpr (kStage > 1) {
2518     if ((tile_K_seqlen + 1) < Tc) {
2519         CP_ASYNC_WAIT_GROUP(0);
2520     }
2521     __syncthreads();
2522 }
2523 } // end outer loop over K seqlen
2524 __syncthreads();
2525
2526 // =====
2527 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
2528 // =====
2529 #pragma unroll
2530 for (int i = 0; i < kValTileSeqLenP; ++i) {
2531     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2532     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2533     #pragma unroll
2534     for (int j = 0; j < kValTileHeadDimV; ++j) {
2535         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2536         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2537         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2538         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2539         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2540         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2541         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2542         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2543     }
2544 }
2545
2546 // =====
2547 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2548 // =====
2549 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2550 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2551 #pragma unroll
2552 for (int i = 0; i < kValTileSeqLenP; ++i) {
2553     #pragma unroll
2554     for (int j = 0; j < kValTileHeadDimV; ++j) {
2555         uint32_t R_Z[2][4];
2556         R_Z[0][0] = R_D[i][j][0];
2557         R_Z[1][0] = R_D[i][j][1];
2558         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2559         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2560         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2561         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2562         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2563         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2564
2565         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2566         if (lane_id % 4 == 0) {
2567             int store_warp_regs_0_Br =
2568                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2569             int store_lane_gmem_0_Br =
2570                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2571             int store_warp_regs_0_d =
2572                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2573             int store_lane_gmem_0_d = store_warp_regs_0_d;
2574             int store_gmem_0_addr_0 = 0_gmem_offset +
2575                 (store_lane_gmem_0_Br + 0) * kHeadDim +
2576                 store_lane_gmem_0_d;

```

```

2577     int store_gmem_0_addr_1 = 0_gmem_offset +
2578         (store_lane_gmem_0_Br + 8) * kHeadDim +
2579         store_lane_gmem_0_d;
2580     // LDST128BITS = reinterpret_cast<float4*>
2581     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2582         *reinterpret_cast<float4*>(&R_Z[0][0]);
2583     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2584         *reinterpret_cast<float4*>(&R_Z[1][0]);
2585 }
2586 }
2587 }
2588 }
2589
2590 // =====
2591 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
2592 // =====
2593
2594 static inline void check(cudaError_t err, const char *msg) {
2595     if (err != cudaSuccess) {
2596         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2597         exit(EXIT_FAILURE);
2598     }
2599 }
2600
2601
2602 static void test_block_reduce(int N) {
2603
2604     srand(42);
2605     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2606     for (int i = 0; i < N; i++)
2607         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2608
2609     // CPU reference: sum of all elements
2610     double ref = 0.0;
2611     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2612
2613     float *d_a, *d_y;
2614     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2615     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2616
2617     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2618     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2619
2620     dim3 block(128);
2621     dim3 grid((N + 127) / 128);
2622     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2623     check(cudaGetLastError(), "blockreduce launch");
2624     check(cudaDeviceSynchronize(), "blockreduce sync");
2625
2626     float result;
2627     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2628
2629     float err = fabsf(result - (float)ref);
2630     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2631         err < 1e-2f ? "PASS" : "FAIL");
2632
2633     free(h_a);
2634     cudaFree(d_a); cudaFree(d_y);
2635 }
2636
2637
2638 static void test_dot(int N) {
2639

```

```

2640 srand(42);
2641 float *h_a = (float *)malloc((size_t)N * sizeof(float));
2642 float *h_b = (float *)malloc((size_t)N * sizeof(float));
2643 for (int i = 0; i < N; i++) {
2644     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2645     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2646 }
2647
2648 // CPU reference
2649 double ref = 0.0;
2650 for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2651
2652 float *d_a, *d_b, *d_y;
2653 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2654 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2655 check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2656
2657 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2658 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2659 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2660
2661 dim3 block(128);
2662 dim3 grid((N + 127) / 128);
2663 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2664 check(cudaGetLastError(), "dot launch");
2665 check(cudaDeviceSynchronize(), "dot sync");
2666
2667 float result;
2668 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2669
2670 float err = fabsf(result - (float)ref);
2671 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2672     err < 1e-2f ? "PASS" : "FAIL");
2673
2674 // ---- Dot Vec4 ----
2675 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2676 dim3 block_v4(32);
2677 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2678 check(cudaGetLastError(), "dot_vec4 launch");
2679 check(cudaDeviceSynchronize(), "dot_vec4 sync");
2680
2681 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2682 float err_v4 = fabsf(result - (float)ref);
2683 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2684
2685 free(h_a); free(h_b);
2686 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2687 }
2688
2689
2690 static void test_relu(int N) {
2691     srand(42);
2692     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2693     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2694     for (int i = 0; i < N; i++)
2695         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2696
2697     float *d_x, *d_y;
2698     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2699     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2700     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2701
2702     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);

```

```

2703 dim3 block256(256);
2704 dim3 grid256((N + 255) / 256);
2705 relu<<<grid256, block256>>>(d_x, d_y, N);
2706 check(cudaGetLastError(), "relu launch");
2707 check(cudaDeviceSynchronize(), "relu sync");
2708 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2709 float max_err = 0.0f;
2710 for (int i = 0; i < N; i++) {
2711     float expected = fmaxf(0.0f, h_x[i]);
2712     float err = fabsf(h_y[i] - expected);
2713     if (err > max_err) max_err = err;
2714 }
2715 printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2716
2717 dim3 block64(64);
2718 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2719 check(cudaGetLastError(), "relu_vec4 launch");
2720 check(cudaDeviceSynchronize(), "relu_vec4 sync");
2721 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2722 max_err = 0.0f;
2723 for (int i = 0; i < N; i++) {
2724     float expected = fmaxf(0.0f, h_x[i]);
2725     float err = fabsf(h_y[i] - expected);
2726     if (err > max_err) max_err = err;
2727 }
2728 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2729
2730 free(h_x); free(h_y);
2731 cudaFree(d_x); cudaFree(d_y);
2732 }
2733
2734
2735 static void test_elementwise(int N) {
2736     srand(42);
2737     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2738     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2739     for (int i = 0; i < N; i++) {
2740         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2741         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2742     }
2743
2744     float *d_a, *d_b, *d_c;
2745     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2746     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2747     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2748     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2749     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2750
2751     dim3 block256(256);
2752     dim3 grid256((N + 255) / 256);
2753     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2754     check(cudaGetLastError(), "eadd launch");
2755     check(cudaDeviceSynchronize(), "eadd sync");
2756     float *h_c = (float *)malloc((size_t)N * sizeof(float));
2757     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2758     float max_err = 0.0f;
2759     for (int i = 0; i < N; i++) {
2760         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2761         if (err > max_err) max_err = err;
2762     }
2763     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2764
2765     dim3 block64(64);

```



```

2766 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2767 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2768 check(cudaGetLastError(), "eadd_vec4 launch");
2769 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2770 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2771 max_err = 0.0f;
2772 for (int i = 0; i < N; i++) {
2773     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2774     if (err > max_err) max_err = err;
2775 }
2776 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2777
2778 free(h_a); free(h_b); free(h_c);
2779 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2780 }
2781
2782
2783 static void test_histogram(int N) {
2784     srand(42);
2785     int BINS = 16;
2786     int *h_hist = (int *)calloc(BINS, sizeof(int));
2787     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2788     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2789     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2790     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2791
2792     int *d_idx, *d_hist;
2793     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2794     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2795     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2796     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2797
2798     dim3 block256(256);
2799     dim3 grid256((N + 255) / 256);
2800     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2801     check(cudaGetLastError(), "histogram launch");
2802     check(cudaDeviceSynchronize(), "histogram sync");
2803     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2804
2805     float max_err = 0.0f;
2806     for (int i = 0; i < BINS; i++) {
2807         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2808         if (err > max_err) max_err = err;
2809     }
2810     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2811
2812     free(h_hist); free(h_hist_ref); free(h_idx);
2813     cudaFree(d_idx); cudaFree(d_hist);
2814 }
2815
2816
2817 static void test_softmax(int N) {
2818     // Use N = blockDim.x so one block processes all elements as one token.
2819     constexpr int NUM_THREADS = 256;
2820     if (N != NUM_THREADS) {
2821         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2822         return;
2823     }
2824
2825     srand(42);
2826     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2827     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2828     for (int i = 0; i < N; i++)

```

```

2829     h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2830
2831 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2832 float max_val = -FLT_MAX;
2833 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2834 double sum_exp = 0.0;
2835 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2836 for (int i = 0; i < N; i++)
2837     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2838
2839 float *d_x, *d_y;
2840 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2841 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2842
2843 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2844
2845 dim3 block(256);
2846 dim3 grid(1); // one token covering all N elements
2847 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2848 check(cudaGetLastError(), "softmax launch");
2849 check(cudaDeviceSynchronize(), "softmax sync");
2850
2851 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2852 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2853
2854 float max_err = 0.0f;
2855 for (int i = 0; i < N; i++) {
2856     float err = fabsf(h_y[i] - h_y_ref[i]);
2857     if (err > max_err) max_err = err;
2858 }
2859 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2860
2861 // ---- Safe Softmax ----
2862 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2863 check(cudaGetLastError(), "safe_softmax launch");
2864 check(cudaDeviceSynchronize(), "safe_softmax sync");
2865 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
2866 max_err = 0.0f;
2867 for (int i = 0; i < N; i++) {
2868     float err = fabsf(h_y[i] - h_y_ref[i]);
2869     if (err > max_err) max_err = err;
2870 }
2871 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2872
2873 // ---- Naive Softmax ----
2874 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2875 check(cudaGetLastError(), "naive_softmax launch");
2876 check(cudaDeviceSynchronize(), "naive_softmax sync");
2877 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
2878 max_err = 0.0f;
2879 for (int i = 0; i < N; i++) {
2880     float err = fabsf(h_y[i] - h_y_ref[i]);
2881     if (err > max_err) max_err = err;
2882 }
2883 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2884
2885 free(h_x); free(h_y); free(h_y_ref);
2886 cudaFree(d_x); cudaFree(d_y);
2887 }
2888
2889 static void test_rms_norm(int N, int K) {
2890
2891

```

```

2892 srand(42);
2893 float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2894 float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2895 for (int i = 0; i < N * K; i++)
2896     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2897 float g = 1.5f; // gain
2898
2899 // CPU reference: y = (x / rms(x)) * g
2900 float epsilon = 1e-5f;
2901 for (int n = 0; n < N; n++) {
2902     double sum_sq = 0.0;
2903     for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
2904     float rms = sqrtf((float)sum_sq / (float)K + epsilon);
2905     for (int k = 0; k < K; k++)
2906         h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
2907 }
2908
2909 float *d_x, *d_y;
2910 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
2911 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
2912 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
2913
2914 dim3 block(128);
2915 dim3 grid(N);
2916 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
2917 check(cudaGetLastError(), "rmsnorm launch");
2918 check(cudaDeviceSynchronize(), "rmsnorm sync");
2919
2920 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2921 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
2922
2923 float max_err = 0.0f;
2924 for (int i = 0; i < N * K; i++) {
2925     float err = fabsf(h_y[i] - h_y_ref[i]);
2926     if (err > max_err) max_err = err;
2927 }
2928 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2929
2930 // ---- RMS Norm Vec4 ----
2931 dim3 block_rv4(32);
2932 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
2933 check(cudaGetLastError(), "rmsnorm_vec4 launch");
2934 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
2935 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
2936 max_err = 0.0f;
2937 for (int i = 0; i < N * K; i++) {
2938     float err = fabsf(h_y[i] - h_y_ref[i]);
2939     if (err > max_err) max_err = err;
2940 }
2941 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2942
2943 free(h_x); free(h_y); free(h_y_ref);
2944 cudaFree(d_x); cudaFree(d_y);
2945 }
2946
2947
2948 static void test_layer_norm(int N, int K) {
2949
2950     srand(42);
2951     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2952     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2953     for (int i = 0; i < N * K; i++)
2954         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

2955 float g = 1.5f, b = 0.3f; // gain and bias
2956
2957 // CPU reference: y = ((x - mean) / std) * g + b
2958 float epsilon = 1e-5f;
2959 for (int n = 0; n < N; n++) {
2960     double sum = 0.0;
2961     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
2962     float mean = (float)sum / (float)K;
2963     double sum_sq = 0.0;
2964     for (int k = 0; k < K; k++) {
2965         float diff = h_x[n * K + k] - mean;
2966         sum_sq += (double)diff * (double)diff;
2967     }
2968     float std = sqrtf((float)sum_sq / (float)K + epsilon);
2969     for (int k = 0; k < K; k++)
2970         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
2971 }
2972
2973 float *d_x, *d_y;
2974 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
2975 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
2976 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
2977
2978 dim3 block(128);
2979 dim3 grid(N);
2980 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
2981 check(cudaGetLastError(), "layernorm launch");
2982 check(cudaDeviceSynchronize(), "layernorm sync");
2983
2984 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2985 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
2986
2987 float max_err = 0.0f;
2988 for (int i = 0; i < N * K; i++) {
2989     float err = fabsf(h_y[i] - h_y_ref[i]);
2990     if (err > max_err) max_err = err;
2991 }
2992 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2993
2994 // ---- Layer Norm Vec4 ----
2995 dim3 block_lv4(32);
2996 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
2997 check(cudaGetLastError(), "layernorm_vec4 launch");
2998 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
2999 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3000 max_err = 0.0f;
3001 for (int i = 0; i < N * K; i++) {
3002     float err = fabsf(h_y[i] - h_y_ref[i]);
3003     if (err > max_err) max_err = err;
3004 }
3005 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3006
3007 free(h_x); free(h_y); free(h_y_ref);
3008 cudaFree(d_x); cudaFree(d_y);
3009 }
3010
3011
3012 static void test_rope(int seq_len, int N) {
3013
3014     int total_pairs = seq_len * N;
3015     int total_elems = total_pairs * 2;
3016     size_t size = (size_t)total_elems * sizeof(float);
3017

```

```

3018 float *h_x = (float *)malloc(size);
3019 float *h_y_ref = (float *)malloc(size);
3020
3021 srand(42);
3022 for (int i = 0; i < total_elems; i++)
3023     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3024
3025 // CPU reference: 2D rotation for each pair
3026 for (int idx = 0; idx < total_pairs; idx++) {
3027     int token_pos = idx / N;
3028     int token_idx = idx % N;
3029     float x1 = h_x[idx * 2];
3030     float x2 = h_x[idx * 2 + 1];
3031     float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3032     float angle = (float)token_pos * theta;
3033     float cos_v = cosf(angle);
3034     float sin_v = sinf(angle);
3035     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3036     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3037 }
3038
3039 float *d_x, *d_y;
3040 check(cudaMalloc(&d_x, size), "rope alloc X");
3041 check(cudaMalloc(&d_y, size), "rope alloc Y");
3042 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3043
3044 dim3 block(256);
3045 dim3 grid((total_pairs + 255) / 256);
3046 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3047 check(cudaGetLastError(), "rope launch");
3048 check(cudaDeviceSynchronize(), "rope sync");
3049
3050 float *h_y = (float *)malloc(size);
3051 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3052
3053 float max_err = 0.0f;
3054 for (int i = 0; i < total_elems; i++) {
3055     float err = fabsf(h_y[i] - h_y_ref[i]);
3056     if (err > max_err) max_err = err;
3057 }
3058 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3059
3060 free(h_x);
3061 free(h_y);
3062 free(h_y_ref);
3063 cudaFree(d_x);
3064 cudaFree(d_y);
3065 }
3066
3067
3068 static void test_mat_transpose(int row, int col) {
3069
3070     size_t size_in = (size_t)row * col * sizeof(float);
3071     size_t size_out = (size_t)col * row * sizeof(float);
3072
3073     float *h_x = (float *)malloc(size_in);
3074     float *h_y_ref = (float *)malloc(size_out);
3075
3076     srand(42);
3077     for (int i = 0; i < row * col; i++)
3078         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3079
3080     // CPU reference: y[j][i] = x[i][j] (row-major)

```

```

3081 for (int i = 0; i < row; i++)
3082     for (int j = 0; j < col; j++)
3083         h_y_ref[j * row + i] = h_x[i * col + j];
3084
3085 float *d_x, *d_y;
3086 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3087 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3088 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3089
3090 dim3 block(16, 16);
3091 dim3 grid((col + 15) / 16, (row + 15) / 16);
3092 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3093 check(cudaGetLastError(), "mattrans launch");
3094 check(cudaDeviceSynchronize(), "mattrans sync");
3095
3096 float *h_y = (float *)malloc(size_out);
3097 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3098
3099 float max_err = 0.0f;
3100 for (int i = 0; i < col * row; i++) {
3101     float err = fabsf(h_y[i] - h_y_ref[i]);
3102     if (err > max_err) max_err = err;
3103 }
3104 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3105
3106 free(h_x);
3107 free(h_y);
3108 free(h_y_ref);
3109 cudaFree(d_x);
3110 cudaFree(d_y);
3111 }
3112
3113
3114 static void test_mat_transpose_padded(int row, int col) {
3115
3116     size_t size_in = (size_t)row * col * sizeof(float);
3117     size_t size_out = (size_t)col * row * sizeof(float);
3118
3119     float *h_x = (float *)malloc(size_in);
3120     float *h_y_ref = (float *)malloc(size_out);
3121
3122     srand(42);
3123     for (int i = 0; i < row * col; i++)
3124         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3125
3126     // CPU reference: y[j][i] = x[i][j] (row-major)
3127     for (int i = 0; i < row; i++)
3128         for (int j = 0; j < col; j++)
3129             h_y_ref[j * row + i] = h_x[i * col + j];
3130
3131     float *d_x, *d_y;
3132     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3133     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3134     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3135
3136     dim3 block(16, 16);
3137     dim3 grid((col + 15) / 16, (row + 63) / 64);
3138     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3139     check(cudaGetLastError(), "mattrans_padded launch");
3140     check(cudaDeviceSynchronize(), "mattrans_padded sync");
3141
3142     float *h_y = (float *)malloc(size_out);
3143     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");

```

```

3144
3145 float max_err = 0.0f;
3146 for (int i = 0; i < col * row; i++) {
3147     float err = fabsf(h_y[i] - h_y_ref[i]);
3148     if (err > max_err) max_err = err;
3149 }
3150 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3151
3152 free(h_x);
3153 free(h_y);
3154 free(h_y_ref);
3155 cudaFree(d_x);
3156 cudaFree(d_y);
3157 }
3158
3159
3160 static void test_sgemv(int M, int K) {
3161
3162     srand(42);
3163     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3164     float *h_x = (float *)malloc((size_t)K * sizeof(float));
3165     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3166     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3167     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3168
3169     // CPU reference: y[m] = sum_k A[m,k] * x[k]
3170     for (int m = 0; m < M; m++) {
3171         double sum = 0.0;
3172         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3173         h_y_ref[m] = (float)sum;
3174     }
3175
3176     float *d_a, *d_x, *d_y;
3177     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3178     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3179     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3180
3181     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3182     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3183
3184     dim3 block(32, 4);
3185     dim3 grid((M + 3) / 4);
3186     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3187     check(cudaGetLastError(), "sgemv launch");
3188     check(cudaDeviceSynchronize(), "sgemv sync");
3189
3190     float *h_y = (float *)malloc((size_t)M * sizeof(float));
3191     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3192
3193     float max_err = 0.0f;
3194     for (int m = 0; m < M; m++) {
3195         float err = fabsf(h_y[m] - h_y_ref[m]);
3196         if (err > max_err) max_err = err;
3197     }
3198     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3199
3200     // ---- SGEMV K32 ----
3201     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3202     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3203     check(cudaGetLastError(), "sgemv_k32 launch");
3204     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3205
3206     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");

```



```

3207
3208 max_err = 0.0f;
3209 for (int m = 0; m < M; m++) {
3210     float err = fabsf(h_y[m] - h_y_ref[m]);
3211     if (err > max_err) max_err = err;
3212 }
3213 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3214
3215 // ---- SGEMV K16 ----
3216 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3217 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3218
3219 int K16 = 16;
3220 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3221 h_x = (float *)malloc((size_t)K16 * sizeof(float));
3222 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3223 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3224 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3225
3226 for (int m = 0; m < M; m++) {
3227     double sum = 0.0;
3228     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3229     h_y_ref[m] = (float)sum;
3230 }
3231
3232 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3233 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3234 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3235
3236 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3237 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3238
3239 dim3 grid_k16((M + 7) / 8);
3240 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3241 check(cudaGetLastError(), "sgemv_k16 launch");
3242 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3243
3244 h_y = (float *)malloc((size_t)M * sizeof(float));
3245 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3246
3247 max_err = 0.0f;
3248 for (int m = 0; m < M; m++) {
3249     float err = fabsf(h_y[m] - h_y_ref[m]);
3250     if (err > max_err) max_err = err;
3251 }
3252 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3253
3254 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3255 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3256 }
3257
3258
3259 static void test_sgemm(int M, int N, int K) {
3260
3261     size_t size_a = (size_t)M * K * sizeof(float);
3262     size_t size_b = (size_t)K * N * sizeof(float);
3263     size_t size_c = (size_t)M * N * sizeof(float);
3264
3265     float *h_a = (float *)malloc(size_a);
3266     float *h_b = (float *)malloc(size_b);
3267     float *h_c_ref = (float *)malloc(size_c);
3268
3269     srand(42);

```

```

3270 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3271 for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3272
3273 float *d_a, *d_b, *d_c;
3274 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3275 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3276 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3277
3278 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3279 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3280
3281 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3282 cublasHandle_t handle;
3283 cublasCreate(&handle);
3284 float alpha = 1.0f, beta = 0.0f;
3285 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3286             &alpha, d_b, N, d_a, K, &beta, d_c, N);
3287 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3288
3289 // Kernel
3290 cudaEvent_t start, stop;
3291 cudaEventCreate(&start);
3292 cudaEventCreate(&stop);
3293
3294 dim3 block(32, 32);
3295 dim3 grid((N + 31) / 32, (M + 31) / 32);
3296 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3297 check(cudaGetLastError(), "sgemm launch");
3298 check(cudaDeviceSynchronize(), "sgemm sync");
3299
3300 float *h_c = (float *)malloc(size_c);
3301 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3302
3303 // Verify
3304 float max_err = 0.0f;
3305 for (int i = 0; i < M * N; i++) {
3306     float err = fabsf(h_c[i] - h_c_ref[i]);
3307     if (err > max_err) max_err = err;
3308 }
3309 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3310
3311 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3312
3313 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3314 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3315 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3316 check(cudaGetLastError(), "sgemm_vec4 launch");
3317 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3318
3319 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3320
3321 max_err = 0.0f;
3322 for (int i = 0; i < M * N; i++) {
3323     float err = fabsf(h_c[i] - h_c_ref[i]);
3324     if (err > max_err) max_err = err;
3325 }
3326 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3327
3328 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3329 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3330 cublasDestroy(handle);
3331 cudaEventDestroy(start);
3332 cudaEventDestroy(stop);

```

```

3333 }
3334
3335
3336 static void test_hgemmm_mma(int M, int N, int K) {
3337
3338     size_t size_a = (size_t)M * K * sizeof(half);
3339     size_t size_b = (size_t)K * N * sizeof(half);
3340     size_t size_c = (size_t)M * N * sizeof(half);
3341
3342     half *h_a = (half *)malloc(size_a);
3343     half *h_b = (half *)malloc(size_b);
3344     half *h_c_ref = (half *)malloc(size_c);
3345
3346     srand(42);
3347     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3348     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3349
3350     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3351     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3352     size_t size_b_t = (size_t)N * K * sizeof(half);
3353     half *h_b_t = (half *)malloc(size_b_t);
3354     for (int n = 0; n < N; n++)
3355         for (int k = 0; k < K; k++)
3356             h_b_t[n * K + k] = h_b[k * N + n];
3357
3358     half *d_a, *d_b, *d_b_t, *d_c;
3359     check(cudaMalloc(&d_a, size_a), "hgemmm alloc A");
3360     check(cudaMalloc(&d_b, size_b), "hgemmm alloc B (cuBLAS)");
3361     check(cudaMalloc(&d_b_t, size_b_t), "hgemmm alloc B_t (kernel)");
3362     check(cudaMalloc(&d_c, size_c), "hgemmm alloc C");
3363
3364     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemmm H2D A");
3365     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemmm H2D B (cuBLAS)");
3366     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemmm H2D B_t (kernel)");
3367
3368     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3369     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3370     cublasHandle_t handle;
3371     cublasCreate(&handle);
3372     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3373     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3374                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3375                 &beta_h, d_c, CUDA_R_16F, N,
3376                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3377     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm D2H ref");
3378
3379     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3380     cudaEvent_t start, stop;
3381     cudaEventCreate(&start);
3382     cudaEventCreate(&stop);
3383
3384     constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3385     size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3386     dim3 block(256);
3387     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3388     hgemmm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3389     check(cudaGetLastError(), "hgemmm launch");
3390     check(cudaDeviceSynchronize(), "hgemmm sync");
3391
3392     half *h_c = (half *)malloc(size_c);
3393     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemmm D2H");
3394
3395     // Verify

```

```

3396 float max_err = 0.0f;
3397 for (int i = 0; i < M * N; i++) {
3398     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3399     if (err > max_err) max_err = err;
3400 }
3401 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3402
3403 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3404 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3405 cublasDestroy(handle);
3406 cudaEventDestroy(start);
3407 cudaEventDestroy(stop);
3408 }
3409
3410
3411 static void test_flash_attn(int seqlen, int head_dim) {
3412     // FlashAttention-2 with split-Q, MMA m16n8k16
3413     int B = 1, H = 8;
3414
3415     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3416
3417     srand(42);
3418     half *h_q = (half *)malloc(sz);
3419     half *h_k = (half *)malloc(sz);
3420     half *h_v = (half *)malloc(sz);
3421     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3422         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3423         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3424         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3425     }
3426
3427     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
3428     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3429     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3430     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3431     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3432     int count = B * H * seqlen * head_dim;
3433     for (int i = 0; i < count; i++) {
3434         ref_q[i] = __half2float(h_q[i]);
3435         ref_k[i] = __half2float(h_k[i]);
3436         ref_v[i] = __half2float(h_v[i]);
3437     }
3438
3439     float scale = 1.0f / sqrtf((float)head_dim);
3440     for (int bi = 0; bi < B * H; bi++) {
3441         for (int qi = 0; qi < seqlen; qi++) {
3442             // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
3443             float smax = -INFINITY;
3444             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3445             for (int kj = 0; kj < seqlen; kj++) {
3446                 float s = 0.0f;
3447                 for (int d = 0; d < head_dim; d++)
3448                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3449                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3450                 S[kj] = s * scale;
3451                 if (S[kj] > smax) smax = S[kj];
3452             }
3453             // softmax
3454             double sum_exp = 0.0;
3455             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3456             float inv_sum = 1.0f / (float)sum_exp;
3457             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3458             for (int d = 0; d < head_dim; d++) {

```

```

3459     double o_acc = 0.0;
3460     for (int kj = 0; kj < seqlen; kj++)
3461         o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3462             ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3463     ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3464 }
3465 free(S);
3466 }
3467 }
3468
3469 half *d_q, *d_k, *d_v, *d_o;
3470 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3471 check(cudaMalloc(&d_k, sz), "fa alloc K");
3472 check(cudaMalloc(&d_v, sz), "fa alloc V");
3473 check(cudaMalloc(&d_o, sz), "fa alloc O");
3474 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3475 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3476 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3477
3478 // Template params for kHeadDim=64, kStage=2
3479 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3480 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3481 constexpr int kMmaTileSeqLenQ = 4;
3482 constexpr int kMmaTileSeqLenK = 1;
3483 constexpr int kMmaTileSeqLenP = 4;
3484 constexpr int kMmaTileHeadDimV = 1;
3485 constexpr int kValTileSeqLenQ = 1;
3486 constexpr int kValTileSeqLenK = 8;
3487 constexpr int kValTileSeqLenP = 1;
3488 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3489
3490 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
3491 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3492 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3493     kStageV * Bc * (kHeadDim + kPadV) +
3494     Bc * (kHeadDim + kPadV)) * sizeof(half);
3495
3496 dim3 block(128);
3497 dim3 grid((seqlen + Br - 1) / Br, B * H);
3498
3499 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3500     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3501     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3502     kStageV, kPadV>
3503     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3504 check(cudaGetLastError(), "fa launch");
3505 check(cudaDeviceSynchronize(), "fa sync");
3506
3507 half *h_o = (half *)malloc(sz);
3508 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3509
3510 float max_err = 0.0f;
3511 for (int i = 0; i < count; i++) {
3512     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3513     if (err > max_err) max_err = err;
3514 }
3515 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3516
3517 free(h_q); free(h_k); free(h_v); free(h_o);
3518 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3519 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3520 }
3521

```

```

3522
3523 int main(int argc, char *argv[]) {
3524     int M = 1024, N = 1024, K = 1024;
3525     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3526
3527     printf("=== notes-v2.cu verification harness ===\n");
3528     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3529     printf("|-----|-----|-----|\n");
3530
3531     test_block_reduce(N);
3532     test_dot(N);
3533     test_relu(1024);
3534     test_elementwise(1024);
3535     test_histogram(1024);
3536     test_softmax(256); // softmax kernel requires N == blockDim.x
3537     test_rms_norm(8, 128);
3538     test_layer_norm(8, 128);
3539     test_rope(8, 128);
3540     test_mat_transpose(256, 256);
3541     test_mat_transpose_padded(256, 256);
3542     test_sgemv(256, 128);
3543     test_sgemm(M, N, K);
3544     test_hgemm_mma(M, N, K);
3545     test_flash_attn(1024, 64);
3546
3547     printf("=== All tests done ===\n");
3548     return 0;
3549 }
3550
3551 // =====
3552 // Quick build & run reference
3553 // =====
3554 // # sm_89 单独编译 + 运行:
3555 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3556 //
3557 // # sm_90a 单独编译 (运行需要 H800 Hopper GPU) :
3558 // nvcc -std=c++20 -O2 -arch=sm_90a -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin

```